A Pattern Language for Production LLM Platforms

Governed Routing, Agent Orchestration, and AI-Native Delivery

Nabeel Khan
iSystematic Inc.
ORCID 0009-0005-5364-914X

Pattern Language Specification, Version 1.1
30 August 2026
doi.org/10.5281/zenodo.22109864
CC BY 4.0

Contents

Abstract **3**

Notation **3**

1. Introduction **4**
1.1 The problem . . . 4
1.2 The central thesis . . . 4
1.3 Contribution . . . 4
1.4 Scope . . . 4
1.5 Non-goals . . . 5

2. Conceptual Model (Normative) **5**
2.1 Boundary . . . 5
2.2 Optimizer . . . 5
2.3 Evidence . . . 5
2.4 Authority . . . 6
2.5 The boundary taxonomy . . . 6
2.6 Three layers, one control path . . . 6
2.7 Figure 1: the pattern language . . . 6
2.8 Figure 2: the boundary, optimizer and evidence model . . . 7
2.9 The Boundary Invariant . . . 7
2.10 The BOE Declaration . . . 8

3. Pattern Derivation (Normative) **9**
3.1 Candidate generation . . . 9
3.2 The four-stage filter . . . 9
3.3 Admission criteria . . . 9
3.4 The derivation applied: admitted and rejected candidates . . . 10
3.5 Evidence levels . . . 10
3.6 Grounding taxonomy . . . 11
3.7 The pattern form . . . 11
3.8 The pattern relationship graph . . . 11

4. The Pattern Catalogue (Normative) **12**
4.1 Infrastructure patterns (IP-1 to IP-5) . . . 12
4.2 Application patterns (AP-1 to AP-5) . . . 14
4.3 Operations patterns (OP-1 to OP-7) . . . 17

4.4 Catalogue summary 20

5. Composition (Normative) 21
5.1 Composition requirements 21
5.2 Event identity 21
5.3 Layer composition 22
5.4 Worked composition 22
5.5 Using the language as an assessment lens 23

6. Applicability (Normative) 23
6.1 Applicability conditions 23
6.2 Environment profiles 24

7. Adoption Sequence 24
7.2 Implementation notes per stage 24

8. Conformance (Normative) 25
8.1 Self-declared conformance 25
8.2 Conformance criteria 25
8.3 The conformance profile 25
8.4 Documented gaps 25
8.5 What a conformance claim does not mean 25

9. Related Work 26
9.1 The Alexandrian pattern tradition 26
9.2 The software design-pattern tradition 26
9.3 Foundation-model and agent pattern catalogues 26
9.4 Machine-learning systems and platform engineering 27
9.5 Governance standards this language does not duplicate 27

10. Limitations and Known Failure Modes 27
10.1 It assumes an institution that values reconstruction above throughput 27
10.2 It governs the architecture, not the models or the practice inside it 27
10.3 The abstraction from mechanism to force is a claim, not a guarantee 27
10.4 It assumes a platform team with the authority to set boundaries centrally 28
10.5 It does not address whether a system should exist 28
10.6 The five characteristic failure modes 28
10.7 Where the language has not been tested 28
10.8 What would invalidate the language 29
10.9 Research questions 29

11. Relationship to Other Frameworks 30

Appendix A. Cross-Corpus Naming Crosswalk (Normative) 30

Appendix B. Publication Provenance 32

12. References 32

Document History 35

About the Author 35

Licence 36

How to attribute this work 36

Pattern Language Specification, Version 1.1

Author: Nabeel Khan
ORCID: 0009-0005-5364-914X
Affiliation: iSystematic Inc.

doi.org/10.5281/zenodo.22109864

Version: 1.1
Date: 30 August 2026
Status: Public technical specification. Self-declared conformance only. Not a certification scheme.
Licence: Creative Commons Attribution 4.0 International (CC BY 4.0)
DOI (cite this): https://doi.org/10.5281/zenodo.22109864
This version: https://doi.org/10.5281/zenodo.22177680

Cite as: Khan, N. A. (2026). *A Pattern Language for Production LLM Platforms: Governed Routing, Agent Orchestration, and AI-Native Delivery. Pattern Language Specification, Version 1.1.* Zenodo. https://doi.org/10.5281/zenodo.22109864

Abstract

A production platform built on large language models makes two kinds of decision, and most of its trouble comes from writing both into one clause. An optimization decision improves an objective: lower latency, lower cost, higher quality, fewer tests run. A boundary decision fixes a constraint that may not be relaxed for any gain: a residency rule, a least-privilege scope, a human-review threshold. When the two share a clause, improving one silently erodes the other, which is why efficiency and accountability are so often reported as a trade.

The contribution of this specification is a cross-layer architectural method for separating non-negotiable constraints from adaptive optimization and binding both to reconstructable evidence, applied identically across model routing, agent orchestration and AI-native delivery. The seventeen patterns are instances of that method rather than the contribution itself.

Each pattern is specified in the classical pattern form and carries three architectural declarations: the **boundary** it fixes, the **optimizer** it frees, and the **evidence** proving the boundary held. Section 3 states the derivation method by which candidates were admitted or rejected, so that the size and membership of the catalogue can be examined rather than assumed. Section 4 is the catalogue. Section 5 states how the patterns compose, including the event identity that makes the three evidence artifacts joinable in fact rather than in aspiration.

The contribution is architectural rather than empirical. Nothing here has been measured. Section 10 states separately what would refute the invariant, the individual patterns and the adoption sequence, and poses six research questions the specification is written to make answerable.

Version 1.1 names two constructs this specification already contained. The central proposition is named **the Boundary Invariant**, and the three architectural declarations required of every pattern are together named **the BOE Declaration**. Both names are supplied at 2.9 and 2.10. Neither carries a trademark, both are offered for use with attribution under this document's licence, and neither changes any requirement: the proposition, its wording and its priority date are those of version 1.0.

Keywords: software architecture; design patterns; pattern language; large language models; LLMOps; agent orchestration; AI governance; platform engineering; evidence architecture; constrained optimization; PEVG; PARA.

Notation

This specification uses the verbal forms of ISO/IEC Directives Part 2:

- **shall** indicates a requirement. An implementation that does not meet it is not conformant.
- **should** indicates a recommendation. Departure requires a recorded rationale.
- **may** indicates a permission.

”Platform” means the set of services through which an institution's applications reach language models, together with the machinery that governs and ships them. ”Boundary” and ”optimizer” carry the meanings fixed in 2.1 and 2.2. ”Evidence artifact” means a record written for later reconstruction, as distinguished from a debugging log. ”Institution” means the accountable legal entity operating the platform. ”Regulated institution” means one whose ability to operate is conditioned on its ability to reconstruct and defend a decision to an external authority.

Every pattern is identified by a stable identifier of the form IP-n, AP-n or OP-n. **The identifiers are normative and shall not be renumbered.** Pattern names are aliases for the identifier. Where this specification, the author's

papers and the companion book series use different names for one construct, all are valid and none is deprecated; Appendix A fixes the equivalences.

Sections 2 through 8 are normative. Sections 1, 9, 10 and 11 are informative, except where they state a requirement using **shall**.

---

1. Introduction

1.1 The problem

A working demonstration of a language model answers a question and turns a pipeline green. A production system in a regulated institution answers the same question under a different set of obligations. The institution must be able to say which model produced an answer and why, what the call cost, what data it touched, whether a human reviewed it, and how the decision would be reconstructed for an auditor a year later. The distance between the demonstration and that obligation is not a matter of scale. It is a change in what the system is for. The demonstration optimizes a response. The production system governs a decision.

The distance is usually described as an engineering gap and treated with more engineering, which is why the treatment so often disappoints. The failure is more commonly a classification error. A platform team writes one clause that both selects the cheapest adequate model and enforces a jurisdictional residency rule. A single prompt both plans a task and decides what may leave the system. A pipeline both learns which tests to skip and holds the tests that may never be skipped. In each case a constraint and an objective are entangled in one mechanism, and the optimizer that improves the objective is therefore free to erode the constraint without anyone deciding that it should. Nothing is misconfigured. The category was wrong before the configuration existed.

1.2 The central thesis

> **A boundary is a clause the optimizer may not cross. Everything else is optimization.**

The language begins from that discipline. Every mechanism in a production platform is classified before it is built: it either improves an objective or it fixes a constraint, and it may not do both. Once the two are separated, the optimizer can be given more freedom rather than less, because the boundary no longer moves when the optimizer pushes against it. The invariant is a design instrument rather than a caution. It is what makes autonomy safe to grant.

The rest of this document uses that proposition rather than restating it.

1.3 Contribution

Stated as five contestable claims, each testable in principle and none tested here.

1. **One invariant across three layers.** Governed routing, agent orchestration and AI-native delivery share a single internal shape, and that shape is the separation of a boundary from an optimizer. Section 2.
2. **A derivation method with an explicit rejection rule.** The boundary test admits a mechanism as a pattern and rejects it as an optimization. Section 3 shows rejected candidates alongside admitted ones, so the criterion can be examined rather than trusted.
3. **Evidence as a first-class architectural output.** Three evidence artifacts at three altitudes, produced by the architecture rather than instrumented afterward, and joined by a required event identity. Sections 2.3 and 5.2.
4. **Authority as a granted, evidenced and revocable quantity.** Autonomy may be raised only as fast as measured evidence justifies, and the mechanism that raises it is the mechanism that records it. OP-2, OP-4.
5. **Economic constraint as a boundary rather than a target.** A spend cap belongs in the same class as a residency rule, and measuring spend is categorically different from bounding it. OP-6, OP-7.

1.4 Scope

In scope: the full request path from entry to response, including which model served it, under what budget, and what evidence the decision left; the composition of agents into workflows, including role separation, workflow structure, oversight and agent authority; and the delivery path by which models, agents, code and policies reach production, including the authority granted to automated actors and the evidence justifying each grant.

doi.org/10.5281/zenodo.22109864

1.5 Non-goals

Naming the boundary of the language is part of the language, because a pattern catalogue used past its scope produces confidence it has not earned.

1. **Model selection, training and evaluation as machine-learning problems.** The language governs how a model choice is bounded and recorded, not how a model is built or which is better. See 10.2.
2. **The decision to deploy at all.** The patterns presume a use case that has already cleared the question of whether the institution should be running it. See 10.5.
3. **Institutional AI governance above the platform.** Board risk appetite, committee structure and supervisory registers sit at a higher altitude and are specified separately in the MESA Framework. See Section 11.
4. **Any specific vendor, framework or serving stack.** The patterns are stated at the level of the force each resolves, and 10.3 states plainly that this abstraction is a claim rather than a guarantee.
5. **Empirical performance claims.** This document reports no benchmark and no measured result. The worked composition in 5.4 is an explicitly fictional illustration.
6. **Legal or regulatory advice.** The document is a structural description. An institution applying it should obtain qualified counsel for the obligations its platform must satisfy.

---

2. Conceptual Model (Normative)

2.1 Boundary

2.1.1 Classification. Every mechanism in a production platform shall be classified, before it is built, as either an optimizer or a boundary. A **boundary** is a mechanism that fixes a constraint which may not be relaxed for any gain in any objective. A mechanism that performs both functions in one clause is not conformant and shall be separated.

2.1.2 Non-crossing. An optimizer shall not relax a boundary at run time for a local gain, under any objective, including an objective the institution values highly.

2.1.3 Governed movement. A boundary may change. It shall change only through the same governed promotion that any policy change follows: reviewed, recorded and reversible. An optimizer may surface a candidate constraint it infers would reduce risk; a candidate is a proposal and not a boundary until it has passed promotion and been recorded. The invariant is therefore not that boundaries are static but that they move only by governance and never by optimization.

2.2 Optimizer

An **optimizer** is a mechanism that improves an objective. Optimizers are the source of a platform's value, and the purpose of drawing boundaries precisely is to give optimizers more room rather than less.

2.2.1 Autonomy. Raising a platform's autonomy shall be performed by widening what the optimizer may do, never by relaxing a boundary. A platform may raise authority as fast as it can prove judgement, and the proof shall be recorded in the same evidence the boundary requires.

2.3 Evidence

Every boundary shall be accompanied by evidence that it held. A boundary asserted without a corresponding evidence artifact is an intention rather than a control, because nothing distinguishes a boundary that was respected from one that was never tested.

2.3.1 The three evidence artifacts. The platform shall produce three artifacts, one per layer: **Routing Evidence**, **Execution Evidence** and **Delivery Evidence**. They are the same idea at three altitudes: a decision, its justification, and proof that it happened. The term *evidence artifact* is used in preference to *record* or *log* deliberately, because the distinction from debugging telemetry is the point of the requirement rather than an incidental property of it.

2.3.2 The reconstruction requirement. An institution shall be able to compose the three artifacts into a single account of a single event, naming who routed what, which agents acted under which contracts and review, and how any resulting change reached production and under whose authority. A layer that operates without producing its evidence artifact is not conformant, irrespective of how well its boundary is drawn. The mechanism that makes composition possible is specified normatively in 5.2.

doi.org/10.5281/zenodo.22109864

2.4 Authority

Authority is the quantity of action a non-human actor may take without a human in the path. It is granted rather than assumed, bounded by a declared artifact, justified by current measured evidence, and revocable. Authority is not a property of an agent's capability; it is a property of the institution's confidence, and the two move on different schedules.

2.5 The boundary taxonomy

Boundaries are treated uniformly by the invariant, but they are not of one kind, and a specification that does not distinguish them invites the objection that ”boundary” has been stretched to mean any constraint the author wished to name. Five classes are distinguished. Every boundary in Section 4 is assigned to exactly one.

Class	Fixes	Typical artifact	Patterns
Data	Where data may go and what may be retained	Residency rule, data-handling class, retention exclusion	IP-1, IP-2, AP-3
Authority	What an actor may do	Capability contract, trust tier, faculty grant	AP-5, OP-2, OP-5
Decision	What must be believed or reviewed before release	Verifier rule, review threshold	AP-1, AP-4
Resource	What may be consumed	Latency target, spend cap	IP-3, OP-7
Process	What must occur before a change takes effect	Promotion gate, never-skip test set, benchmark gate	IP-5, OP-1, OP-3, OP-4

Two patterns fix an evidence obligation rather than a constraint on behaviour, and are classified separately as **evidence-constituting**: IP-4 and OP-6. They are patterns because they make other boundaries enforceable, which is the second limb of the admission test in 3.3.

2.6 Three layers, one control path

The three layers are not a stack of independent systems. They are one control path observed at three altitudes.

Layer	The question it answers	Boundary	Optimizer	Evidence artifact
Infrastructure	Which model may serve this request, under what latency, cost and risk budget, and how is that decision recorded?	Routing policy; residency and risk constraints	Learned router selecting the cheapest adequate model	**Routing Evidence**
Application	How are specialized agents composed into a workflow that can be inspected, reviewed and improved without becoming opaque?	Capability contract; human-review threshold	Planner performing task decomposition	**Execution Evidence**
Operations	How do models, agents, code and policy reach production on a platform that holds authority by degrees rather than all at once?	Delivery guardrail; trust tier	Delivery pipeline selecting tests and acting within its authority	**Delivery Evidence**

2.6.1 Independence of mechanism. The patterns describe the structure of the work, not the mechanism that implements it. They hold whether a workflow is built from autonomous agent loops, from deterministic pipelines with retrieval and structured execution, or from a hybrid. A deterministic step still needs a declared boundary on what it may touch, an evidence artifact recording what it did, and a review tier when its stakes are high.

2.7 Figure 1: the pattern language

Boundaries descend from governance. Evidence ascends to reconstruction. The event identity is what makes the ascent possible, and is the reason the evidence plane is drawn as one plane rather than three.

doi.org/10.5281/zenodo.22109864

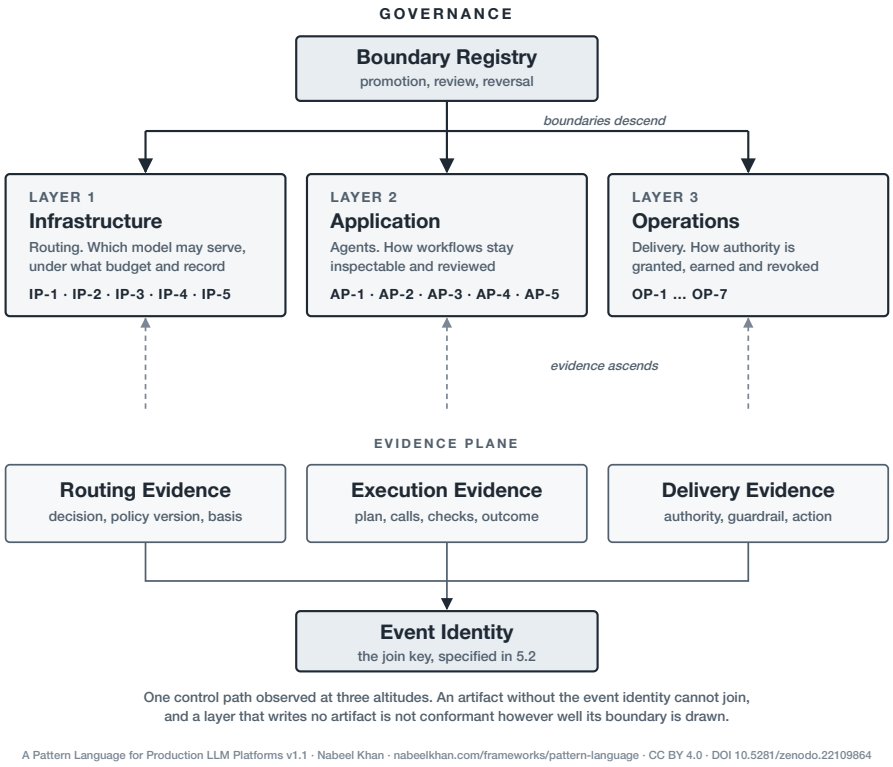


Figure 1. The pattern language: boundaries descend from governance into three layers, evidence ascends into one plane, and a single event identity joins the three artifacts.

2.8 Figure 2: the boundary, optimizer and evidence model

Every pattern in Section 4 is an instance of this figure. The three declarations under each pattern name its boundary, its optimization space and its evidence, in that order.

2.9 The Boundary Invariant

2.9.1 The name. The proposition stated at 1.2, that a boundary is a clause the optimizer may not cross and everything else is optimization, is named **the Boundary Invariant**.

Version 1.0 of this specification stated the proposition and did not name it. Version 1.1 supplies the name and changes nothing else. The wording at 1.2 is unchanged, the requirements at 2.1, 2.2 and 2.3 are unchanged, no pattern is affected, and no conformance claim made against version 1.0 is disturbed. **The proposition's priority date is the publication date of version 1.0, 26 August 2026.** The name is later than the proposition and does not restate it.

2.9.2 Why a name was needed. An unnamed proposition can be used and cannot be cited. It cannot be taught without paraphrase, disputed without quotation, or extended by anyone other than its author, and a paraphrase of a proposition is not the proposition. Naming it does not strengthen it. Naming it makes it possible for a third party to attack it precisely, which is the condition under which it might turn out to be wrong, and 10.8 states separately what would falsify it.

2.9.3 What the name denotes, and what it does not. The Boundary Invariant is the proposition, not this specification and not the catalogue. A reader may accept the invariant and reject every one of the seventeen patterns; the patterns are instances of the method the invariant licenses, and 1.3 states that as the contribution. A reader may equally implement the patterns without accepting the invariant, and will get a catalogue rather than a language.

2.9.4 No trademark is claimed. The Boundary Invariant is **deliberately unmarked**. It is offered for use, teaching, assessment, extension and criticism by anyone, with attribution, under this document's CC BY 4.0 licence, and no permission beyond that licence is required.

doi.org/10.5281/zenodo.22109864

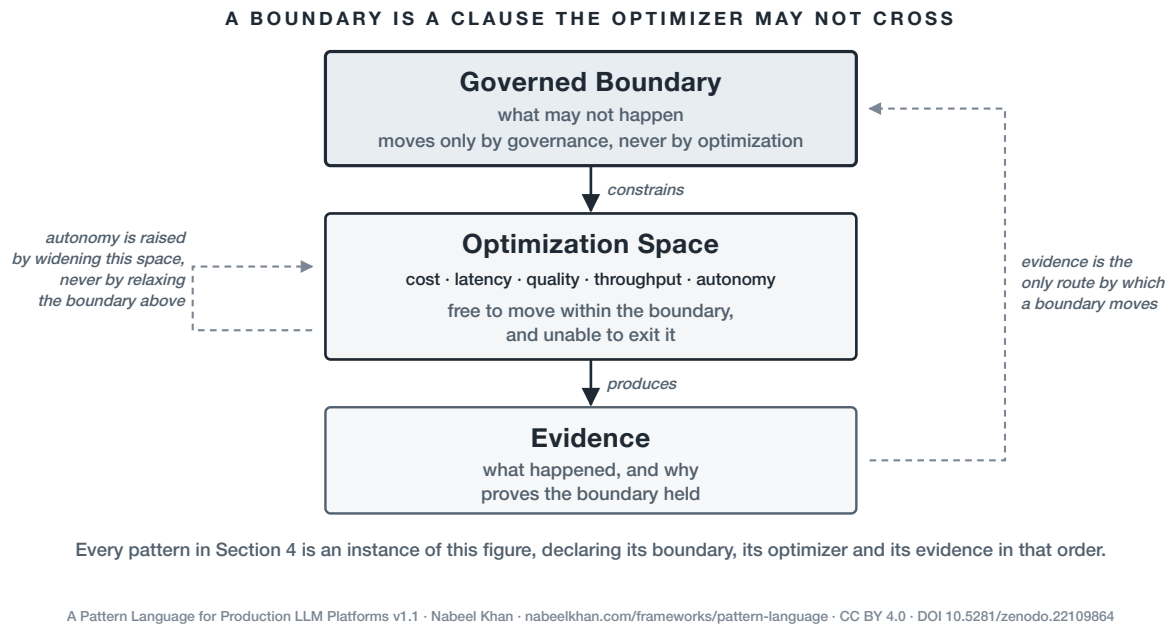


Figure 2. The boundary, optimizer and evidence model. Autonomy is raised by widening the optimization space, never by relaxing the boundary above it.

The reason is strategic rather than generous. A mark on a proposed law of architecture suppresses the citation the proposition needs in order to become one, and the defensibility of the name rests on a dated, citable publication record rather than on a symbol. Section 11.1 states the same rule for the rest of the engineering family.

2.10 The BOE Declaration

2.10.1 The name. The three architectural declarations required of every pattern by 3.7.1, the **B**oundary a pattern fixes, the **O**ptimizer it frees, and the **E**vidence proving the boundary held, are together named the **BOE Declaration**.

As with 2.9, version 1.0 required all three of all seventeen patterns and did not name the triple. Nothing is added here. 3.7.2 already states that a pattern which cannot state all three has not passed 3.3, and that requirement is unchanged.

2.10.2 It is a convention, not a requirement. The BOE Declaration is the name of a document convention. Its value is that a convention with a name can be copied, and the copying is the point: a control specified outside this document that declares its boundary, its optimizer and its evidence in this shape produces evidence joinable with evidence produced under these patterns. Two controls that emit evidence in the same shape can be composed. Two that do not are merely adjacent, however well each is drawn.

2.10.3 It generalizes past a pattern. Any control an institution operates may carry a BOE Declaration, whether or not the control is a pattern in this catalogue. A deployment gate, a validation record, a residency attestation and a vendor screening record can each be stated as one boundary fixed, one optimizer freed, and the evidence that the boundary held. Where those controls belong to a framework outside this specification, Section 11 states which document governs.

2.10.4 What it does not do. A BOE Declaration is not a control, an assurance, or a claim that the boundary in fact held. It states which boundary a control fixes and which artifact would show that it held. Whether the artifact exists is the evidence requirement at 2.3, and whether it was produced is a matter of fact rather than of declaration.

2.10.5 No trademark is claimed. The BOE Declaration is **deliberately unmarked**, on the same terms and for the same reason as 2.9.4. A document convention nobody may adopt without permission is a document convention nobody adopts.

doi.org/10.5281/zenodo.22109864

3. Pattern Derivation (Normative)

A catalogue that does not say how its members were chosen invites the reasonable question of why there are seventeen rather than twelve or forty-one. This section states the method, shows candidates that were rejected as well as admitted, and classifies the evidence behind each pattern so that a reader can distinguish what is established from what is this author's synthesis.

3.1 Candidate generation

Pattern candidates were generated from four sources:

1. **Recurring production failure modes** observed or documented in language-model platforms, particularly failures of reconstruction rather than of output quality.
2. **Architectural control requirements** that a platform must satisfy before an institution can answer for its behaviour.
3. **Governance constraints** imposed on regulated institutions by supervisors, internal policy or contractual obligation.
4. **Optimization mechanisms whose safe operating region requires an explicit boundary**, which is the class most often missed, because the mechanism appears benign until the optimizer is given room.

3.2 The four-stage filter

Each candidate passed through four stages in order:

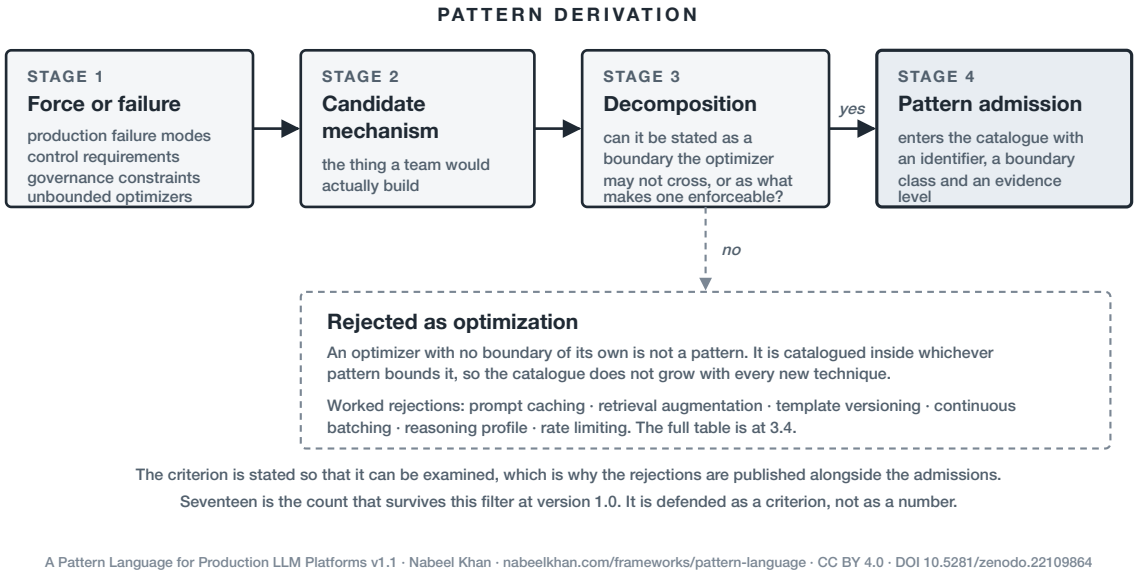


Figure 3. The four-stage derivation filter. A candidate that decomposes into an optimizer with no boundary is rejected and catalogued inside whichever pattern bounds it.

A candidate that cannot be decomposed into a boundary and an optimizer at stage three does not reach stage four. A candidate that decomposes into an optimizer with no boundary is an optimization and is catalogued inside whichever pattern bounds it, not as a pattern of its own.

3.3 Admission criteria

3.3.1 A candidate shall be admitted only where all three hold:

1. It is required by one of the three layers to move from a call to a governed event, from an agent to an accountable workflow, or from a change to a defended deployment.
2. It is grounded in a body of published work or in citable practice.
3. It can be stated as a boundary the optimizer may not cross, **or** as the mechanism that makes such a boundary enforceable.

doi.org/10.5281/zenodo.22109864

3.3.2 A mechanism failing the third criterion is an optimization inside a pattern and shall not be catalogued as a pattern.

3.3.3 Where a pattern's mechanism is established in the literature but its governance addition is not, the Known uses field shall state both halves separately. A pattern whose Known uses field claims more than the record supports damages the catalogue more than a pattern that admits its uses are emerging.

3.4 The derivation applied: admitted and rejected candidates

The table below is a sample of the filter's operation, not the complete working. It is included so that the rejection rule can be examined against cases where it excluded something plausible.

Candidate	Force	Boundary	Optimizer	Verdict
Model routing	Cost against quality	Residency, permitted model class	Learned router	**Admitted, IP-1**
Capability declaration	Ungoverned authority growth	Capability contract	Agent behaviour in contract	**Admitted, AP-5**
Spend cap	Runaway autonomous cost	Budget ceiling	Quality-cost utility	**Admitted, OP-7**
Test selection	Pipeline speed against risk	Never-skip set	Learned selection	**Admitted, OP-3**
Prompt caching	Cost and latency	None. Cache validity is a correctness property, not a constraint that may never be relaxed for gain	Cache policy	Rejected. Optimization inside IP-1
Retrieval augmentation	Answer grounding	None of its own. What a workflow may read is fixed by the capability contract	Retrieval strategy	Rejected. Subsumed by AP-5
Prompt or template versioning	Reproducibility	Subsumed. The versioned-artifact-under-promotion requirement is already IP-1 and IP-5	Template refinement	Rejected. Subsumed
Continuous batching	Throughput	None. Bounded by the declared latency target	Batch policy	Rejected. Optimization inside IP-3
Reasoning Profile	Planner calibration by business line	None. It parameterizes a planner rather than constraining it	Decomposition behaviour	Rejected. Parameterization of AP-1. See A.4
Rate limiting	Resource protection	Subsumed. A rate limit is a resource boundary already generalized by OP-7	Admission policy	Rejected. Subsumed by OP-7

3.4.1 Why seventeen. Seventeen is the count that survives 3.3 across the three layers at the date of this version. The number carries no significance and is not defended as either exhaustive or minimal; 10.7 states that limitation directly. What is defended is the criterion. A reader who believes an eighteenth pattern is required is invited to state it in the form of 3.6 and show that it passes 3.3, and a reader who believes one of the seventeen fails 3.3 is invited to say which limb it fails.

3.4.2 A standing constraint on growth. A mechanism that fits an existing pattern shall be documented under that pattern rather than admitted as a new one. The invariant is intended to do the work that additional patterns would otherwise be recruited to do. A catalogue that grows with each new serving technique has become a taxonomy of everything the author has encountered, which is the failure state this rule exists to prevent.

3.5 Evidence levels

Every pattern in Section 4 carries an evidence level. The levels describe the evidence for the pattern **as specified here**, which is usually the combination of an established mechanism with a governance addition, and not the evidence for the mechanism alone.

Level	Meaning
E0	Conceptual only. No external implementation evidence for the mechanism or the combination.
E1	Published mechanism. The underlying mechanism exists in published research or documented practice; the governance addition specified here is not separately documented.

doi.org/10.5281/zenodo.22109864

Level	Meaning
E2	Documented implementation. The combined mechanism, including its governance addition, has been implemented and documented.
E3	Independent production evidence. The pattern has evidence from an implementation outside the author's own work.

3.5.1 No pattern in this catalogue is at E3. That is the single most important fact about the evidence base and is stated here rather than left to be inferred from seventeen separate fields. Two patterns reach E2. The remaining fifteen are E1. The catalogue is therefore an architectural synthesis over established mechanisms, and it should be read as one.

3.6 Grounding taxonomy

Because the language is connected to the author's own books and frameworks, a reader is entitled to know how much of it validates itself against that corpus. Four grounding roles are distinguished, and the catalogue's position on each is stated once here rather than repeated seventeen times.

Grounding role	Position across the catalogue
External grounding	Present for fifteen of seventeen patterns, cited per pattern in Known uses. OP-6 and OP-7 rest on established practice rather than on a citable study.
Author-originated synthesis	The boundary-optimizer decomposition, the three-artifact evidence model, the event identity and the adoption sequence are this author's synthesis and are not drawn from a prior source.
Author implementation	AP-1 and OP-1 reach E2. AP-1 is implemented in a production agent platform operated by the author, including the separate policy-enforcement point at the workflow output that AP-1.1 requires, with the two decisions recorded separately.
Independent implementation	**None currently.** No pattern has been evaluated by an implementer unconnected to the author.

3.7 The pattern form

3.7.1 Required fields. Every pattern shall carry, in this order: **Context**, the situation in which it applies; **Problem**, the difficulty it resolves; **Forces**, the tensions the solution must trade; **Solution**, stated normatively; the three architectural declarations **Boundary**, **Optimizer** and **Evidence**; **Consequences**; **Known uses**, the published or citable grounding, or an explicit statement that documented uses are emerging; and **Evidence level** per 3.5.

3.7.2 The three architectural declarations are the operational core of the form. A pattern that cannot state all three has not passed 3.3.

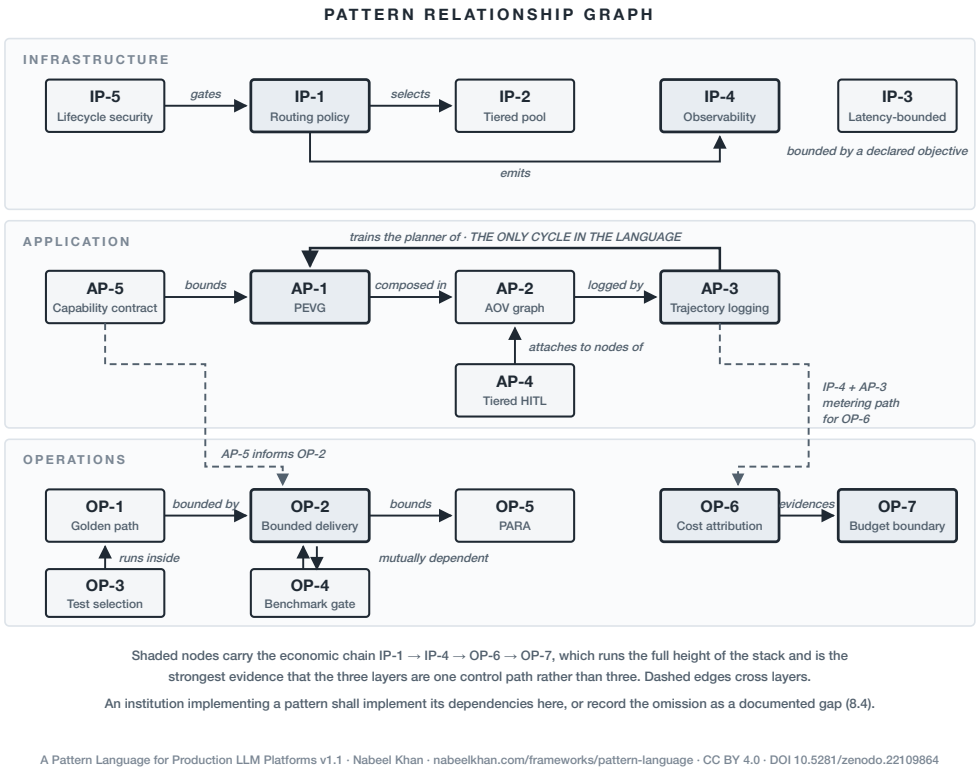
3.8 The pattern relationship graph

A pattern language is more than a list, and this subsection is where the claim is discharged. The relationships below are the ones that carry load: where one pattern supplies the artifact, evidence or authority another depends on.

3.8.1 Three properties of the graph are worth naming. The application layer contains a **cycle**, AP-1 to AP-2 to AP-3 and back to AP-1, which is the loop by which a workflow improves from its own structural record; it is the only cycle in the language. The **economic chain** runs the full height of the stack, from a routing decision to a spend cap, which is the strongest evidence that the three layers are one control path rather than three. And **OP-2 and OP-4 are mutually dependent**, because authority without a current benchmark is unjustified and a benchmark without an authority consequence is a number in a report.

3.8.2 An institution implementing a pattern shall implement the patterns it depends on in this graph, or record the omission as a documented gap under 8.4.

doi.org/10.5281/zenodo.22109864



Shaded nodes carry the economic chain IP-1 → IP-4 → OP-6 → OP-7, which runs the full height of the stack and is the strongest evidence that the three layers are one control path rather than three. Dashed edges cross layers.

An institution implementing a pattern shall implement its dependencies here, or record the omission as a documented gap (8.4).

A Pattern Language for Production LLM Platforms v1.1 · Nabeel Khan · nabeelkhan.com/frameworks/pattern-language · CC BY 4.0 · DOI 10.5281/zenodo.22109864

Figure 4. The pattern relationship graph. Shaded nodes carry the economic chain across the full height of the stack; the application layer contains the only cycle in the language.

4. The Pattern Catalogue (Normative)

Seventeen patterns in three layers. Each carries the three architectural declarations of 2.8, a boundary class from 2.5, and an evidence level from 3.5. Full name equivalences across the papers and the book series are in Appendix A.

4.1 Infrastructure patterns (IP-1 to IP-5)

The infrastructure layer turns a model call into a governed event.

IP-1. Governed Routing Policy

Context. An institution runs a pool of models differing in capability, cost, latency and data-handling guarantees, and a stream of requests differing in sensitivity and difficulty.

Problem. A routing rule embedded in application code cannot be reviewed, versioned or audited, and it mixes a business constraint with an optimization choice.

Forces. The institution wants the cheapest adequate model for each request. It also requires that some requests never leave a jurisdiction and never reach a particular model class, and that the reason for every route can be reconstructed.

Solution. Routing shall be expressed as a declarative, versioned policy artifact separate from the application. The policy shall state the hard constraints as boundaries, specifically residency and permitted model classes per data tier, and shall expose the soft objective, cost and latency minimization subject to a quality floor, to a learned router [14][15][16]. The policy shall be promoted through change control and shall not be edited in place.

Boundary *(Data)*. Residency and permitted model class per data tier. **Optimizer.** A learned router choosing the cheapest adequate model. **Evidence.** Routing Evidence: the policy version that decided the route, per IP-4.

Consequences. The routing decision becomes inspectable and governable. The policy artifact is the unit an auditor reviews.

doi.org/10.5281/zenodo.22109864

Known uses. Learned routing across a heterogeneous model pool is documented as an inference-efficiency mechanism [14][15]. Separating the hard policy from the learned objective is the governance addition the regulated setting requires. **Evidence level: E1.**

IP-2. Tiered Model Pool

Context. Requests vary by orders of magnitude in difficulty, and models vary by orders of magnitude in cost.

Problem. Serving every request from the strongest model wastes budget. Serving every request from the cheapest model fails the hard ones.

Forces. Quality, cost and latency pull against one another, and the right balance differs per request.

Solution. The pool shall be organized into capability tiers, and the IP-1 policy shall select a tier per request, escalating only where a cheaper tier's confidence is inadequate [16][17]. Each tier shall carry an attached data-handling class, so that a tier is also a trust class.

Boundary *(Data)*. The data-handling class attached to each tier. **Optimizer.** Per-request tier selection and escalation. **Evidence.** The selected tier and the escalation reason, carried in Routing Evidence.

Consequences. Cost tracks difficulty rather than worst case. The tier becomes the natural place to attach a data-handling constraint.

Known uses. Uncertainty-based routing escalates from a cheaper to a stronger model only where confidence is inadequate [16], which is the tier-escalation behaviour this pattern governs. **Evidence level: E1.**

IP-3. Adaptive Latency-Bounded Serving

> **Naming note.** This pattern was named *SLO-Aware Speculative Decoding* in the author's papers. It is renamed here at the level of the force it resolves, because a pattern named after a serving technique cannot outlive that technique, which is the durability requirement stated in 10.3. Speculative decoding is the current principal implementation, not the pattern.

Context. Latency is a budget the institution commits to, not a value it observes after the fact.

Problem. Generation latency varies with request shape, and a fixed decoding strategy meets the target on some requests and misses it on others.

Forces. Throughput and tail latency trade against each other, and the acceptable trade differs by service-level objective.

Solution. The serving strategy shall be bound to a declared service-level objective and shall adapt to request shape rather than being applied uniformly. *Example implementation:* speculative decoding, with the draft strategy adapted to the latency target [18], long-context variants for new request shapes [19], and distributed serving where the pipeline allows [20]. Any strategy meeting the declared objective satisfies this pattern.

Boundary *(Resource)*. The committed latency target. **Optimizer.** The serving strategy, tuned to request shape. **Evidence.** The target, the strategy selected and the achieved latency, carried in Routing Evidence.

Consequences. The latency target becomes a declared boundary the serving layer optimizes toward rather than an emergent property observed afterward. Because the pattern is stated at the force level, a successor technique replaces the implementation without restating the pattern.

Known uses. Service-level-objective-aware speculative decoding adapts the draft strategy to a latency target [18], long-context variants carry the technique into new request shapes [19], and the field is surveyed comprehensively [20]. **Evidence level: E1.**

IP-4. Decision-Grade Observability

Context. An institution must reconstruct why a particular model answered a particular request, possibly long after the fact.

doi.org/10.5281/zenodo.22109864

Problem. Conventional service telemetry records that a call happened and how long it took. It does not record the decision that selected the model or the basis for it.

Forces. Storage and privacy limit what can be retained. Accountability requires that the decision be reconstructable.

Solution. Routing Evidence shall be emitted for every routing decision, capturing the inputs to the decision, specifically the request class, the applicable policy version and the candidate tiers; the decision itself; and its justification. It shall carry the event identity of 5.2. Payloads shall be excluded where policy requires. The artifact is evidence and shall not be treated as a debugging log.

Boundary *(Evidence-constituting)*. The decision and the basis that must be recorded. **Optimizer.** Payload retention within the privacy policy. **Evidence.** This pattern constitutes the Routing Evidence artifact.

Consequences. The audit question becomes answerable from the artifact alone. Observability stops being instrumentation and becomes part of the governed artifact set.

Known uses. The recasting of routing telemetry as evidence rather than as a debugging log is the governance addition this pattern makes. Documented uses that record the routing decision and its basis for audit, distinct from conventional service telemetry, are emerging. **Evidence level: E1.**

IP-5. Router Lifecycle Security

Context. The router decides where sensitive requests go, which makes it both a high-value target and a single point of governance.

Problem. A router that can be changed without a gate can be made to route sensitive traffic to the wrong place, silently.

Forces. Operational agility wants fast router changes. Security and compliance want every change reviewed.

Solution. The router shall be treated as a governed artifact with a promotion pipeline and a conformance gate, and its lifecycle shall be treated as an attack surface to be defended [21]. A router version shall reach production the way a regulated change does: reviewed, recorded and reversible. The gate shall apply to the IP-1 policy artifact as well as to the router itself.

Boundary *(Process)*. The promotion and conformance gate. **Optimizer.** The router's internal improvement behind the gate. **Evidence.** The promotion record: version, approver, gate result, and the reversal path.

Consequences. A change to where sensitive traffic flows cannot happen without leaving evidence.

Known uses. The router's life-cycle vulnerabilities are documented as an attack surface across adversarial and backdoor threat classes [21], which is the threat a promotion-and-conformance gate is placed to govern. **Evidence level: E1.**

4.2 Application patterns (AP-1 to AP-5)

The application layer composes agents into workflows an institution can stand behind.

AP-1. Planner, Executor, Verifier, Generator *Alias:* ***PEVG*** *(Book 2, Chapter 3). The acronym appears in neither paper.*

Context. A task requires more than one model call: a plan, one or more tool actions, a check, and a final response.

Problem. A single undifferentiated agent that plans, acts, checks and answers in one step cannot be reasoned about, reviewed or improved part by part. When it fails there is no seam to open.

Forces. A single agent is simpler to build and faster to run. Accountability requires that the step which decides what may leave the system be separable from the step that produced it, so that each can be measured and bounded on its own.

Solution. The agent shall be separated into four roles with four declared contracts. A **planner** decomposes the task into an ordered set of steps with explicit dependencies. An **executor** performs tool actions under a capability

doi.org/10.5281/zenodo.22109864

contract (AP-5). A **verifier** checks each result before it is believed. A **generator** produces the final response from what survived the check, and shall not assert anything the verifier did not pass [22][23].

AP-1.1 Two output boundaries, not one. The verifier holds the **epistemic boundary**: what may be *believed*. It does not thereby hold the **operational boundary**: what may be *disclosed or acted upon*. A claim can be true, correctly verified, and still forbidden to leave the system, because disclosure is governed by data classification, authorization and policy rather than by correctness. An implementation shall therefore place a policy-enforcement point at the workflow's output in addition to the verifier, and the two shall be separately recorded. Collapsing them is the characteristic error this subsection exists to prevent: a verifier that is asked to enforce disclosure policy is a single clause performing two functions, which fails 2.1.1.

Boundary *(Decision)*. The verifier's rule on what may be believed, and the policy-enforcement rule on what may leave. **Optimizer.** Each role, measured and improved on its own. **Evidence.** Execution Evidence: the plan, the tool calls, the verifier's decisions and the disclosure decision, per AP-3.

Consequences. Failure becomes locatable, because each role owns a distinct class of error. The planner becomes trainable in isolation from the roles it directs, which is the property AP-3 exists to supply data for.

Known uses. Modular software-engineering agents separate planning, execution and verification into distinct sub-agents [23], and in-the-flow agentic optimization trains a planner separately from the roles it directs [22]. A production agent platform operated by the author separates the two boundaries of AP-1.1, placing a policy-enforcement point at the workflow output in addition to the verifier and recording the two decisions separately. **Evidence level: E2.**

AP-2. Activity-on-Vertex Workflow Graph *Alias: **AOV graph**, also the agent graph (Book 2, Chapter 5).*

> **Naming note.** Unlike IP-3 and OP-3, this pattern was not renamed. ”Activity-on-vertex” names a mathematical structure rather than a technology, and a structure does not become obsolete when a framework does.

Context. A workflow connects several agents with dependencies and opportunities for parallelism.

Problem. A workflow buried in imperative code cannot be inspected, reasoned about or refined without reading the code.

Forces. Imperative code is expressive and immediate. A workflow the institution must review and refine has to be an artifact it can read, version and reason about without executing.

Solution. The workflow shall be modelled as an activity-on-vertex graph in which vertices are agent invocations and edges are dependencies, with the work residing in the vertices rather than on the edges, and shall be serialized as inspectable configuration [24][25].

Boundary *(Process)*. The declared dependency structure. **Optimizer.** Parallelism discovered within the graph. **Evidence.** The serialized graph version that executed, referenced from Execution Evidence.

Consequences. The workflow becomes an artifact that can be reviewed, versioned and automatically refined. Parallelism becomes a property of the graph rather than a hidden detail of the code, and the longest dependency chain sets the floor on achievable runtime.

Known uses. Modularized agentic workflow automation defines workflows as activity-on-vertex graphs and refines them during execution [24], and automated agentic workflow generation represents the workflow as a searchable, code-represented graph refined by Monte Carlo tree search [25]. **Evidence level: E1.**

AP-3. Structure-Only Trajectory Logging

Context. Improving a planner requires the record of what past runs did.

Problem. Logging the full inputs and outputs of every step is a privacy and storage liability inside a regulated institution.

Forces. A richer log trains a better planner and reconstructs more of what happened. Every payload retained is a liability. The record must carry the structure without carrying the content.

Solution. Each trajectory shall be logged as structure: the plan, the sequence of tool calls, the verifier's corrections and the outcome, linked as one example and carrying the event identity of 5.2, with sensitive payloads excluded.

doi.org/10.5281/zenodo.22109864

The four elements shall be linked rather than stored independently, because a plan without its outcome teaches nothing and an outcome without its plan cannot be attributed [22].

Boundary *(Data)*. What may be logged, with payloads excluded. **Optimizer.** A planner trained from the structural record. **Evidence.** This pattern constitutes the Execution Evidence artifact.

Consequences. The institution gains a training signal and an audit trail without retaining the content that would make either a liability. The retention boundary is explicit rather than incidental.

Known uses. In-the-flow agentic optimization trains a planner from the structural record of prior runs [22]. The payload-exclusion boundary is the addition the regulated setting requires, and documented uses of the two together are emerging. **Evidence level: E1.**

AP-4. Tiered Human-in-the-Loop

Context. Some agent actions are reversible and low-stakes. Others move money or make a regulated statement.

Problem. Reviewing everything is impossible, and reviewing nothing is indefensible.

Forces. Human attention is finite and throughput pressure is constant. Some actions are too consequential to release unreviewed. Review must be rationed by stakes rather than spent uniformly.

Solution. Human review shall be designed as a runtime layer with declared tiers, at minimum auto-approve, lightweight review and full supervisory review, with a default tier per node of the AP-2 graph and escalation as stakes rise [26][27].

AP-4.1 Confidence is not a review threshold. Review thresholds shall be based on an evaluated **decision-risk signal**, which may incorporate model confidence but shall not treat raw model confidence as sufficient evidence of correctness. Three quantities are routinely conflated and shall be kept distinct: **model confidence**, a property of a generation process and frequently miscalibrated; **decision confidence**, an assessment of whether the evidence assembled supports the conclusion; and **risk threshold**, an institutional statement of what is at stake if the conclusion is wrong. A tier keyed to raw model confidence alone will route confidently-wrong outputs past review, which is the failure mode most likely to survive testing and appear first in production.

Boundary *(Decision)*. The per-node decision-risk threshold at which a human shall review. **Optimizer.** Review effort tracking risk rather than volume. **Evidence.** The tier applied, the signal that selected it, and the reviewer identity, in Execution Evidence.

Consequences. Review effort tracks risk. Below the institution's bar a human shall look, regardless of throughput pressure, and the threshold shall not be lowered to clear a backlog.

Known uses. Human-in-the-loop orchestration for agentic use cases describes review as an explicit runtime layer with confidence-driven escalation [26], building on a body of human-in-the-loop machine-learning practice that formalizes tiered oversight [27]. **Evidence level: E1.**

AP-5. Capability Contract

Context. A population of agents grows, and no single person can say what each agent may touch.

Problem. Without a declared boundary per agent, capability grows faster than the institution can govern it.

Forces. Agents are cheap to add and each new one widens what the population can do. Ungoverned capability growth is precisely what an institution cannot defend. Every agent must arrive with its authority declared and bounded.

Solution. Every agent shall declare a capability contract stating what it reads, what it produces, what actions it may take, and the boundary it may not cross [23]. Registration in the agent registry shall be conditional on the contract, and an agent absent from the registry shall not run.

Boundary *(Authority)*. The declared capability the agent may not exceed. **Optimizer.** Agent behaviour within the contract. **Evidence.** The registry entry and the per-invocation record of the contract in force.

Consequences. The contract becomes the unit of governance for the agent population, and least privilege becomes declarable rather than aspirational. The characteristic anti-pattern it defeats is the ambient agent: a capability that

doi.org/10.5281/zenodo.22109864

accreted from a useful prompt, answers to no contract, and surfaces only when someone asks what touched a customer's data.

Known uses. Modular software-engineering agents scope each sub-agent to a declared role and toolset [23]. The contract-as-registration-gate framing is the governance addition, and documented uses in regulated deployments are emerging. **Evidence level: E1.**

4.3 Operations patterns (OP-1 to OP-7)

The operations layer ships models, agents, code and policy on one platform that holds authority by degrees.

OP-1. Golden Path

Context. Every team in an institution makes the same delivery decisions, and the institution wants those decisions made once and reused.

Problem. Documented best practice that is easier to leave than to follow is not governance. It is advice.

Forces. Teams want freedom in their delivery choices. The institution wants its accumulated judgement reused. The supported path must be the path of least resistance rather than a document a team is free to ignore.

Solution. The institution's accumulated judgement shall be encoded as a golden path: a supported route from idea to production whose safe way is also the easy way, with enforced defaults and a bounded region of team choice [11][28].

Boundary *(Process)*. The enforced delivery defaults. **Optimizer.** The bounded region of team choice. **Evidence.** Delivery Evidence: the path taken, the defaults enforced, and any exception granted.

Consequences. The path is governance only because leaving it is harder than walking it. A golden path that is merely documented has not been adopted.

Known uses. The golden path originates in platform practice at Spotify as the opinionated and supported path to build something [11] and is treated systematically in the platform-engineering literature [28]. **Evidence level: E2.**

OP-2. Policy-Bounded Agentic Delivery *Aliases:* ***Delivery Guardrail*** *(the boundary artifact) and* ***trust tier*** *(the authority scale), Book 3, Chapters 4 and 5.*

Context. Agents begin to act inside the delivery pipeline, not only to suggest.

Problem. An agent that can act in the pipeline without a declared bound inherits authority the institution cannot defend.

Forces. Autonomy in the pipeline is what makes an agent useful. Unbounded autonomy is precisely what the institution cannot account for. Authority must be granted in degrees rather than assumed.

Solution. Every agent action in the pipeline shall be bounded by a delivery guardrail: a declarative, versioned, promotable artifact stating what the agent may propose, what it may execute, what requires a human, what is forbidden, and the evidence it shall record [29][30]. Authority shall be granted through explicit trust tiers, at minimum suggestion, supervised action, and scope-limited autonomy. Promotion between tiers shall be evidentiary and human-signed; **demotion shall be automatic and immediate**, because the cost of a slow withdrawal is the incident that was not stopped.

Boundary *(Authority)*. The trust tier and the delivery guardrail. **Optimizer.** Agent behaviour inside the granted tier. **Evidence.** Delivery Evidence: the tier in force, the guardrail version, the action taken, and the authority under which it was taken.

Consequences. Autonomy becomes something the platform grants and records rather than something an agent assumes.

Known uses. AI-augmented continuous integration and delivery introduces autonomous decisions into the pipeline [29][30]. The explicit trust-tier bound is the governance addition, and documented uses in regulated pipelines are emerging. **Evidence level: E1.**

doi.org/10.5281/zenodo.22109864

OP-3. Risk-Bounded Adaptive Test Selection

> **Naming note.** This pattern was named *Reinforcement-Framed Adaptive Testing* in the author's papers. As with IP-3, the name is moved to the force level: the pattern is the never-skip floor beneath a learned selection, not the particular learning method. Reinforcement framing is the current principal implementation.

Context. Running every test on every change is slow. Skipping tests blindly is dangerous.

Problem. A fixed test policy cannot adapt to which changes actually carry risk.

Forces. Running every test is slow and running fewer is faster. Some tests guard against failures too costly to risk. Selection may therefore be learned only above a floor that is never learned away.

Solution. Test selection shall be adaptive to the assessed risk of a change, subject to a never-skip set declared per service as a risk profile and placed outside the optimizer's reach. *Example implementation:* framing selection as a reinforcement-learned decision process over the pipeline [31]. Any selection method respecting the declared floor satisfies this pattern.

Boundary *(Process)*. The never-skip test set. **Optimizer.** The learned or heuristic selection around it. **Evidence.** Delivery Evidence: the tests selected, the tests skipped, and the never-skip set version in force.

Consequences. Test effort tracks risk rather than habit, and the floor remains defensible under examination.

Known uses. Reinforcement-framed workflow optimization treats CI/CD test and stage selection as a learned decision process [31]. The declared never-skip floor is the boundary the regulated setting adds. **Evidence level: E1.**

OP-4. Benchmark Before Authority

Context. An agent is proposed for a role in the delivery pipeline.

Problem. Granting authority on the strength of a demonstration invites failure the first time the agent meets a case the demonstration did not cover.

Forces. A demonstration is quick and persuasive. Authority granted on it fails on the first uncovered case. A trust tier must rest on measured performance against real failures.

Solution. An agent shall be measured against a benchmark of real failures for its task before it is granted any authority under OP-2, and shall be re-measured as it changes. The trust tier shall be bound to a **current** benchmark so that authority can fall as well as rise [32][33].

OP-4.1 Authority eligibility is not a benchmark score. A benchmark measures performance on the distribution it samples. Authority is a claim about behaviour outside that distribution, and the two are not the same quantity. A system may score highly and remain unsuitable for authority because the benchmark does not capture distribution shift, adversarial input, privilege escalation, rare catastrophic failure, or interaction effects with other agents. Authority shall therefore require all four of the following, and a benchmark score alone shall not be sufficient:

1. **Benchmark performance** at or above the declared threshold for the task.
2. **Boundary compliance**, evidenced across the benchmark run: no attempt to exceed the capability contract, and no reliance on a boundary being unenforced.
3. **Failure analysis**, naming the classes of failure observed and the classes the benchmark could not exercise.
4. **Currency**, meaning the evidence postdates the agent's most recent material change.

Boundary *(Process)*. The four-part gate on any authority increase. **Optimizer.** Authority raised against measured evidence. **Evidence.** The benchmark result, the boundary-compliance record, the failure analysis and its date, referenced from the OP-2 tier grant.

Consequences. The gate result becomes the evidence that justifies a trust tier, which is what turns OP-2's boundary into something an institution can defend. Because currency is required, authority decays rather than persisting by default.

Known uses. Benchmarks of large-language-model tools against real continuous-integration failures supply the performance measure [32], and studies of agents acting on CI/CD configurations report the frequency and success

doi.org/10.5281/zenodo.22109864

such a benchmark would gate on [33]. The four-part eligibility test is the governance addition. **Evidence level: E1.**

OP-5. Perception, Action, Reasoning, Adaptation *Alias: **PARA**, the PARA operating model (Book 3, Chapter 8).*

Context. Operational agents must do more than alert. They must observe, act, reason about the result, and adjust.

Problem. An operating model that names only perception and action omits the reasoning that justifies an action and the adaptation that closes the loop.

Forces. A two-part perceive-and-act loop is simple and common. An institution needs the justification behind an action recorded and the loop to improve from its results. Reasoning and adaptation must be named as faculties in their own right.

Solution. Operational agents shall be structured around four faculties, each carrying a distinct authority type: **Perception**, read-only access to system signals, emitting structured observations; **Reasoning**, read access to observations and runbooks, emitting a plan and touching nothing; **Action**, the sole authority to change production, and only through enumerated, policy-authorized operations; and **Adaptation**, write access to institutional knowledge and no write access to production. Adaptation shall be bounded by the same guardrails as Action, because an agent that could both act and rewrite the record of its action could launder its own mistakes into institutional memory. Each agent's faculty, allowed actions, forbidden actions, governing guardrail and success metrics shall be recorded in a single registry entry.

Boundary *(Authority)*. The guardrails on Action and on Adaptation. **Optimizer.** The reasoning and adaptation loop within them. **Evidence.** Delivery Evidence: the observation, the plan and its justification, the action taken, and the knowledge updated.

Consequences. The Reasoning faculty is where the agent's justification is recorded, which makes the operating model compatible with the reconstruction requirement of 2.3.2. Authority that reads the world and authority that changes it are bounded and audited apart.

Known uses. Perception-action loops are established in AI-native platform and site-reliability practice [34]. The explicit Reasoning and Adaptation faculties are the extension this operating model adds, and documented uses of the full four-part model are emerging. **Evidence level: E1.**

Naming note. The fourth faculty is **Adaptation**. Sources rendering it as Reflection are in error.

OP-6. Per-Team Cost Attribution

Context. Model and accelerator spend is real, shared and easy to lose track of.

Problem. Cost that is not attributed to the team that incurred it cannot be governed and will grow without an owner. A shared fleet produces one bill; an accountable platform produces many.

Forces. Shared spend is easy to pool and attribution adds accounting the shared model avoids. Spend without an owner cannot be governed.

Solution. Spend shall be attributed to the team that caused it, reusing the Routing Evidence of IP-4 and the Execution Evidence of AP-3 through the shared event identity of 5.2, rather than building a parallel metering path. The attribution shall be **complete**, covering idle held capacity and not only active compute; **granular** enough to act on, reaching the service and workload rather than stopping at the team boundary; and **timely**, surfaced while the spend is recent enough to change the next decision.

Boundary *(Evidence-constituting)*. Attribution of spend to its owner. **Optimizer.** Cost reused as a control signal. **Evidence.** The attribution record, derived from the existing evidence artifacts rather than independently metered.

Consequences. The institution gains a control surface for spend that reuses existing evidence. Attribution does not reduce the bill; it identifies who can.

Known uses. Per-team and per-workload cost attribution is established FinOps practice. Documented uses that attribute spend from the same routing and trajectory evidence an LLM platform already emits are emerging. **Evidence level: E1.**

---

doi.org/10.5281/zenodo.22109864

OP-7. Budget as Boundary

Context. Capability and autonomy both cost money, and the marginal value of an additional call, agent step or retry falls while its marginal cost and risk do not.

Problem. An optimizer told only to raise quality will spend without limit, because nothing in its objective stops it. Cost attribution measures spend but does not bound it: a dashboard reports what was spent, while a boundary decides what may be.

Forces. Quality improves with spend. Budgets are finite. An unbounded autonomous system can consume a quarter's compute on a single hard request.

Solution. A spend cap shall be treated as a hard boundary rather than a soft target. The platform may optimize a utility trading quality against cost, risk and delay, and the budget shall be the clause that utility may not cross. Where a request, an agent run or a team reaches its cap, the system shall degrade deliberately, to a cheaper tier, a halt or an escalation, and shall not spend past the limit. The cap shall be set and changed by governed policy under 2.1.3, with the OP-6 attribution records as its evidence.

OP-7.1 The pattern stated as constrained optimization. The invariant has a standard form in this pattern, and stating it makes the distinction from a soft target precise. Let `x` denote a platform action or policy, `U` a utility over quality and speed, `C` realized cost, `B` the budget boundary, `R` a risk measure with ceiling `R_max`, and `D` the set of permitted data destinations:

$$\begin{aligned} \text{maximize} \quad & U(x) \\ \text{subject to} \quad & C(x) \le B \\ & R(x) \le R_{\text{max}} \\ & D(x) \in \mathcal{D}_{\text{allowed}} \end{aligned}$$

The three constraints are boundaries; `U` is the optimization space of Figure 2. A soft target adds cost as a penalty term inside `U`, which permits the optimizer to buy its way past the limit whenever quality gains outweigh the penalty. That is precisely the boundary-crossing 2.1.2 forbids. The distinction between a constraint and a weighted term in the objective is the entire content of this pattern, and it generalizes: every boundary class in 2.5 can be written as a constraint of this kind, which is why the invariant holds uniformly across the three layers.

Boundary *(Resource)*. The spend cap per request, run or team. **Optimizer.** The quality-versus-cost utility inside the cap. **Evidence.** The cap in force, the spend against it, and every deliberate degradation taken at the limit.

Consequences. Spend becomes governable in the same shape as every other boundary. The pattern closes the economic loop that OP-6 opens.

Known uses. Per-request and per-workload spend caps are established practice in cost-governed serving. Documented uses that treat the cap as a hard boundary with deliberate degradation, rather than as a soft target with an alert, are emerging. **Evidence level: E1.**

4.4 Catalogue summary

ID	Name	Layer	Boundary class	Evidence level
IP-1	Governed Routing Policy	Infrastructure	Data	E1
IP-2	Tiered Model Pool	Infrastructure	Data	E1
IP-3	Adaptive Latency-Bounded Serving	Infrastructure	Resource	E1
IP-4	Decision-Grade Observability	Infrastructure	Evidence-constituting	E1
IP-5	Router Lifecycle Security	Infrastructure	Process	E1
AP-1	Planner, Executor, Verifier, Generator	Application	Decision	**E2**
AP-2	Activity-on-Vertex Workflow Graph	Application	Process	E1
AP-3	Structure-Only Trajectory Logging	Application	Data	E1
AP-4	Tiered Human-in-the-Loop	Application	Decision	E1
AP-5	Capability Contract	Application	Authority	E1

doi.org/10.5281/zenodo.22109864

ID	Name	Layer	Boundary class	Evidence level
OP-1	Golden Path	Operations	Process	**E2**
OP-2	Policy-Bounded Agentic Delivery	Operations	Authority	E1
OP-3	Risk-Bounded Adaptive Test Selection	Operations	Process	E1
OP-4	Benchmark Before Authority	Operations	Process	E1
OP-5	Perception, Action, Reasoning, Adaptation	Operations	Authority	E1
OP-6	Per-Team Cost Attribution	Operations	Evidence-constituting	E1
OP-7	Budget as Boundary	Operations	Resource	E1

5. Composition (Normative)

5.1 Composition requirements

5.1.1 The three evidence artifacts of 2.3.1 shall be joinable. An institution shall be able to reconstruct a single event across all three layers without manual correlation. The mechanism is specified in 5.2.

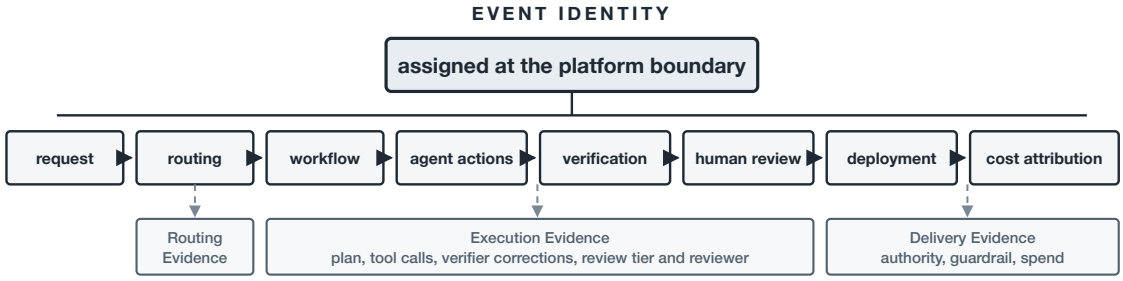
5.1.2 The verifier of AP-1 shall be applied at the workflow's output and not only at each step, and the review tier of AP-4 shall be applicable to the composite action and not only to individual nodes. The reason is stated in 10.6 under agent collusion.

5.1.3 The identity used for cost attribution under OP-6 shall be the event identity of 5.2. A parallel attribution identity defeats the reuse the pattern depends on.

5.2 Event identity

”Joinable” is an aspiration until the join key is specified. This subsection makes it a requirement.

5.2.1 The event identity. Every request entering the platform shall be assigned an **event identity**: an identifier unique within the institution's retention horizon, generated at the platform boundary and never reassigned. It shall be propagated through the full control path:



Four rules make the join hold in practice, and each addresses a way a nominally complete evidence set fails to answer a causal question:

1. Every evidence artifact shall carry the event identity. An artifact written without it cannot join, and therefore does not satisfy 2.3.2.
2. Asynchronous delegation, including agent to agent, shall carry the identity of the immediate parent event, so a fan-out is distinguishable from a sequence.
3. A retry shall keep the identity of the attempt it repeats and be distinguished by an attempt ordinal, because retries cluster where the join is most needed.
4. Where a provider does not propagate the identity, the platform shall record the correlation on its own side. The obligation relocates; it does not lapse.

Retention periods shall be consistent across the three artifacts. The shortest member sets the reconstruction horizon for the whole set.

A Pattern Language for Production LLM Platforms v1.1 · Nabeel Khan · nabeelkhan.com/frameworks/pattern-language · CC BY 4.0 · DOI 10.5281/zenodo.22109864

Figure 5. Event identity propagation, the artifacts that attach to it, and the four rules that make the join hold in practice.

5.2.2 Carriage. Every evidence artifact shall carry the event identity. An artifact written without it does not satisfy 2.3.2, because an artifact that cannot be joined cannot contribute to a reconstruction.

doi.org/10.5281/zenodo.22109864

5.2.3 Causal parentage. Where work is delegated asynchronously, including agent-to-agent delegation, the child shall carry both the event identity and the **identity of the immediate parent event**. Parentage is what allows a reconstruction to distinguish a sequence from a fan-out, and its absence is the most common reason a nominally complete evidence set cannot answer a causal question.

5.2.4 Retries. A retry shall carry the same event identity as the attempt it repeats and shall be distinguishable from it by an attempt ordinal. Assigning a new event identity to a retry breaks the join at the point where it is most often needed, because retries cluster around failures.

5.2.5 Provider boundaries. Where a call crosses into a third-party provider that does not propagate the identity, the platform shall record the correlation on its own side of the boundary. The absence of provider support is not an exemption from 2.3.2; it relocates the obligation to the caller.

5.2.6 Retention. The institution shall declare a retention period for each evidence artifact, and the periods shall be consistent across the three. An artifact retained for a shorter period than its siblings silently shortens the reconstruction horizon of the whole set to the shortest member, which is a property institutions routinely discover only when a reconstruction is requested.

5.2.7 Partial availability. Where one artifact is unavailable, a reconstruction shall be reported as partial and shall name the missing artifact. A reconstruction presented as complete when a member is absent is worse than an acknowledged gap, because it invites reliance the evidence does not support.

5.3 Layer composition

The three layers compose along the control path of 2.6. The pattern relationship graph of 3.8 states which patterns supply artifacts to which. An institution implementing a pattern shall implement its graph dependencies or record the omission under 8.4.

5.4 Worked composition

The composition below is an illustrative construction using a fictional institution, Nebula Financial. It was built to exhibit how the patterns compose, which means it cannot test them. It deploys no system, runs no benchmark and reports no measurement.

The trigger. A transaction is flagged as potentially fraudulent, and an analyst asks the platform to investigate. An event identity is assigned at the platform boundary.

Infrastructure. The governed routing policy (IP-1) recognizes regulated customer data and constrains the request to a permitted capability tier (IP-2) inside the required jurisdiction. The router optimizes only within that constraint. Routing Evidence (IP-4) is written naming the policy version, the candidate tiers and the basis for the route, carrying the event identity. The latency commitment on the interactive path is met by a serving strategy bound to the declared objective (IP-3), and the policy version that produced the route is one the promotion gate approved (IP-5).

Application. The task exceeds a single call and enters the application layer as a workflow. A planner decomposes the investigation. An executor gathers transaction history under a capability contract (AP-5) forbidding any state-changing action. A verifier checks the assembled case against the institution's disclosure rules, and a separate policy-enforcement point decides what may leave (AP-1, per AP-1.1). A generator drafts the finding from what survived both. The workflow is an activity-on-vertex graph (AP-2), and independent evidence-gathering steps run in parallel because no edge connects them. Execution Evidence (AP-3) captures the plan, the tool calls, the verifier's corrections and the outcome without retaining customer payloads, under the same event identity. Because the finding may lead to a regulated action, that node carries a full-supervisory-review tier selected by decision-risk rather than raw model confidence (AP-4). A human signs before anything leaves.

Operations. The investigation exposes a gap in the institution's own controls, and the fix is a change to the platform itself. It enters the operations layer through the governed pipeline along a golden path (OP-1). An agent proposes the remediation, bounded by a delivery guardrail (OP-2) that forbids it from acting beyond a suggestion until a human raises its trust tier. The pipeline runs the never-skip set alongside the adaptive selection around it (OP-3), and the proposing agent had passed the four-part eligibility gate (OP-4) before receiving even suggestion authority. The operational agents that watched the deployment and updated the runbooks held distinct PARA faculties (OP-5). Delivery Evidence is written, and the spend the episode incurred is attributed to its owning team from the same artifacts (OP-6), inside a cap the utility could not cross (OP-7).

doi.org/10.5281/zenodo.22109864

The closing observation. From three artifacts joined on one event identity, the institution can reconstruct who routed what, which agents investigated it under which contracts and review, and how the fix reached production and under whose authority. That reconstruction is the deliverable. The text the models generated along the way is close to incidental, which is the point the language exists to make.

5.5 Using the language as an assessment lens

The catalogue doubles as a diagnostic. Each pattern reduces to a question with a determinate answer, and the answer set is more informative than any aggregate score.

The per-pattern question. For each of the seventeen: is the boundary this pattern fixes stated explicitly as an artifact, separate from the optimizer it bounds, versioned and promoted rather than edited in place, and does its evidence artifact exist?

Three scoring rules. These bind any assessor, and departure from any of them invalidates the result.

1. **Assess against the running system, not the design document.** A boundary that exists in an architecture diagram and not in the deployed control path is not a boundary.
2. **Where a mechanism performs both functions, record it as absent.** A clause that both optimizes and constrains fails 2.1.1, and recording it as partially present conceals exactly the failure the language exists to surface.
3. **Require the evidence artifact before crediting the pattern.** A pattern whose evidence is not produced is not implemented, however faithfully its mechanism is described.

Output. A per-pattern implementation status across the seventeen carrying its as-of date, together with the applicable set from Section 6. The gap between the two is the remediation scope, and Section 7 gives its order.

---

6. Applicability (Normative)

Earlier statements of this language required all seventeen patterns of a regulated institution. That was too strong, and it is corrected here. A pattern that governs a mechanism an institution does not operate is not a gap; it is out of scope. A regulated institution shall implement every **applicable** pattern, not automatically every pattern.

6.1 Applicability conditions

A pattern is applicable where its condition holds. Where the condition does not hold, the pattern is out of scope and shall be recorded as such rather than as an unimplemented gap.

Pattern	Applicable when
IP-1	The platform reaches more than one model or model class
IP-2	Models differ materially in capability, cost or data-handling class
IP-3	The platform commits to a latency objective
IP-4	Any routing or model-selection decision is made
IP-5	The routing policy or router can be changed after deployment
AP-1	A task spans more than one model call
AP-2	A workflow connects more than one agent with dependencies
AP-3	Trajectories are retained for improvement or reconstruction
AP-4	Automated decisions carry consequence for a person or the institution
AP-5	More than one agent exists, or any agent may take action
OP-1	More than one team delivers onto the platform
OP-2	Automated actors act within the delivery path
OP-3	Test selection varies by change
OP-4	Any authority is granted to an automated actor
OP-5	Agents operate the platform rather than only serve requests

doi.org/10.5281/zenodo.22109864

Pattern	Applicable when
OP-6	Spend is shared across more than one team or workload
OP-7	Inference cost is variable, or any actor may spend autonomously

6.2 Environment profiles

The applicability conditions determine the set. The environment determines how strictly the set binds.

Environment	Binding
Startup, early stage	Applicable patterns are recommended. IP-1 and OP-1 should be implemented from the outset, because retrofitting a routing policy and a golden path is materially harder than starting with them
Enterprise, unregulated	Applicable patterns in the infrastructure layer, and AP-5, shall be implemented. The remainder are recommended
Regulated institution	**Every applicable pattern shall be implemented.** The boundaries are obligations rather than choices. Non-applicability shall be recorded with its condition and reviewed when the platform changes

6.2.1 An institution shall re-evaluate applicability whenever it adds a model class, an agent with action authority, an automated actor in the delivery path, or a new spending actor. Each of those four changes brings previously inapplicable patterns into scope, and the review is the point at which that is noticed rather than discovered.

---

7. Adoption Sequence

A platform rarely adopts the language at once. The stages below are the safe sequence in which to grant a platform power. The ordering is normative for regulated institutions under 6.2.

- **Stage 0.** A single model behind an application. No routing, no evidence.
- **Stage 1.** A routing layer (IP-1 to IP-5) that turns a model call into a governed event.
- **Stage 2.** An agent layer (AP-1 to AP-5) that composes calls into inspectable, reviewed workflows.
- **Stage 3.** A governance layer that makes contracts, evidence artifacts and review tiers first-class across both preceding layers, joined by the event identity of 5.2.
- **Stage 4.** An autonomous platform (OP-1 to OP-7) that raises agent authority by degrees, each increase earned against evidence and bounded by budget.

7.1 A platform that reaches Stage 4 without passing through Stage 3 has built autonomy it cannot account for. Stage 3 shall not be skipped or deferred on the ground that the evidence can be added later, because evidence added later is reconstruction.

7.2 Implementation notes per stage

Stage 1. Express the routing policy as an artifact and take it out of application code. Stand up Routing Evidence before the learned router, not after, because a router deployed ahead of its evidence produces decisions that cannot be reviewed. The characteristic blocker is an inventory problem: no complete list exists of the paths by which applications currently reach models, and shadow integrations surface only once the governed path is available and something continues to bypass it.

Stage 2. Separate the agent into its four roles before optimizing any of them. Serialize the workflow as a graph before attempting to refine it. Define the trajectory exclusions with the institution's privacy function before the first artifact is written, because a record that should never have existed is a liability the platform carries forward.

Stage 3. Make contracts, evidence and review tiers first-class across both preceding layers. This is the stage most often deferred and least often recovered. Its completion test is 5.1.1: an arbitrary past event, selected by the requester rather than nominated by the platform team, reconstructed across all three artifacts.

doi.org/10.5281/zenodo.22109864

Stage 4. Grant authority only against a current four-part gate (OP-4), and build the demotion path before the promotion path. Set the spend cap before the autonomy that will test it. The characteristic blocker is organizational rather than technical: the trust tier is a governance decision, and a platform team that holds it alone will find the tier questioned at the first incident.

---

8. Conformance (Normative)

8.1 Self-declared conformance

Conformance under this specification is a self-declared architectural claim unless independently assessed. It indicates implementation against the requirements of this document. It does not imply regulatory compliance, certification, or assurance by the author. No accredited body assesses against this specification, and none is envisaged.

8.2 Conformance criteria

A platform conforms when all of the following hold and are evidenced:

1. Every mechanism in scope is classified as a boundary or an optimizer, and no mechanism performs both functions in one clause (2.1.1).
2. No optimizer relaxes a boundary at run time, and every boundary change is a recorded, reviewed, reversible act (2.1.2, 2.1.3).
3. All three evidence artifacts are produced, each carrying the event identity, and the three join into one account of one event (2.3.2, 5.1.1, 5.2).
4. Every pattern applicable under Section 6 is implemented, with its boundary stated explicitly, its optimizer identified and its evidence produced.
5. Any authority granted to an automated actor is bound to a current four-part gate and is revocable (OP-2, OP-4).
6. Where an applicable pattern is not implemented, the omission is recorded as a documented gap under 8.4 rather than asserted as compliance.

8.3 The conformance profile

A conformance claim shall be published as a profile rather than as a single assertion, because an undifferentiated claim is exactly the opaque assurance this language exists to replace. The profile shall carry:

Profile field	Content
Environment	The profile from 6.2
Applicable set	The patterns in scope under 6.1, with their conditions
Implemented	Per-pattern status across the applicable set
Version	The version of this specification claimed against
Evidence date	The as-of date of the assessment
Exceptions	Documented gaps per 8.4

8.3.1 A profile without an evidence date is expired by default. A profile without the per-pattern detail is not a conformance claim under this specification.

8.4 Documented gaps

A gap is an applicable pattern that is not implemented. Each shall be recorded with the pattern identifier, the reason, a named owner, and a review date. A gap recorded honestly is compatible with a conformance profile; a gap concealed by an aggregate score is not.

8.5 What a conformance claim does not mean

It is not a certification. It is not a statement of regulatory compliance; compliance is determined by supervisors against their own instruments. It is not a performance claim; nothing in this specification has been measured.

doi.org/10.5281/zenodo.22109864

9. Related Work

This section follows the specification rather than preceding it, so that the language can be read on its own terms before it is positioned against others. Three traditions bear on it, and the claim under test is narrow in each case: that the tradition supplies a form or a mechanism this language uses, and leaves open a question this language answers.

Every work cited was verified against the publisher record, the ACM or ACL digital library, or the arXiv abstract record before it entered this document.

9.1 The Alexandrian pattern tradition

The pattern form originates with Christopher Alexander. *A Pattern Language: Towns, Buildings, Construction* [1] presents two hundred and fifty-three patterns, each stating a recurring problem in a context and a solution stated so that it can be applied a million times over without ever being applied the same way twice. *The Timeless Way of Building* [2] supplies the theory beneath it: that a pattern is a relationship among forces in a context, and that a language is the set of such relationships arranged so that the larger patterns give context to the smaller.

Three properties are inherited and stated as requirements in 3.7. A pattern shall name its context. A pattern shall state the forces in tension, because a solution presented without its forces is an instruction rather than a pattern and cannot be reasoned about when the forces change. And a language shall be more than a list, which is the claim 3.8 discharges.

What the tradition does not supply is a selection criterion for what belongs in a catalogue. Alexander's criterion is the felt quality of the resulting environment, which is not testable in this setting. The criterion adopted here is the boundary test of 3.3, and the rejected candidates in 3.4 show it excluding plausible mechanisms rather than merely admitting chosen ones.

9.2 The software design-pattern tradition

Design Patterns: Elements of Reusable Object-Oriented Software [3] catalogues twenty-three patterns of object-oriented design in a uniform template of intent, motivation, applicability, structure, consequences and known uses. *Pattern-Oriented Software Architecture* [4] extends the form upward, spanning architectural patterns, design patterns and idioms in one system.

Two inheritances are direct. The uniform template is adopted, with the fields required by 3.7. The **known uses** field is adopted with the discipline the tradition imposes: a pattern is credible in proportion to the uses that can be cited for it, and where no published use exists the field shall say so. The evidence levels of 3.5 formalize what that tradition leaves to prose.

What that tradition addresses is a different altitude. Its patterns govern the internal structure of a program: how objects are composed, how a subsystem is decoupled, how a call is indirected. The patterns here govern a control path crossing several systems and several disciplines, and the consequences they trade are not maintainability and coupling but cost, latency, defensibility and authority. The template transfers. The forces do not.

9.3 Foundation-model and agent pattern catalogues

The genre has a current generation, and this language is positioned against it rather than in ignorance of it.

The **Agent Design Pattern Catalogue** [5] collects eighteen architectural patterns for foundation-model based agents, with an analysis of the context, forces and trade-offs of each and a decision model for their selection. A companion line documents design patterns for large-language-model-based multi-agent systems in software engineering, organized by the quality attribute each pattern serves [6]. Work on GenAI-native systems, presented at Onward! 2025, states design principles and architectural patterns for robustness and adaptivity at the level of an individual system [7]. A practitioner catalogue systematizes twenty-one agentic patterns as building blocks for intelligent agents [8].

These four share an altitude. They describe the internals of an agent or of a multi-agent system: its planning, memory, tool use, reflection and coordination. They answer how to build a capable agent well, and they answer it with more depth on that question than this language contains.

doi.org/10.5281/zenodo.22109864

The question left open is what surrounds the agent. None specifies the routing policy the agent serves under, the residency constraint that policy encodes, the evidence its actions leave, the budget it may spend, or the delivery pipeline that grants it authority. This language specifies those under one invariant applying identically at all three layers. A team would use an agent-internal catalogue to build a good agent and this cross-layer language to make a platform of such agents accountable. The two are complementary, and the differentiator is not a longer list of patterns but the separation of a boundary from an optimizer applied across layers the prior catalogues treat one at a time or not at all.

9.4 Machine-learning systems and platform engineering

The machine-learning systems literature established, well before language models, that the model is a small part of a production system and that the surrounding machinery carries most of the risk. *Hidden Technical Debt in Machine Learning Systems* [9] names boundary erosion, entanglement, hidden feedback loops and undeclared consumers as the characteristic failures, and the entanglement it describes is the same category error 1.1 identifies, observed one generation earlier. The MLOps literature [10] supplies the definitional and architectural framing for the delivery machinery the operations layer governs. Neither states a rule for deciding which of a system's mechanisms may be learned and which may not, which is the decision this language exists to make.

The platform-engineering record supplies the operations layer's premises directly, including the golden path as the supported route from idea to production [11], and it is cited per pattern rather than in aggregate.

9.5 Governance standards this language does not duplicate

An institution adopting this language and no management-system standard will have an architecture and no control catalogue. ISO/IEC 42001:2023 [12] specifies the certifiable requirements of an AI management system and its Annex A controls. The NIST AI Risk Management Framework [13] specifies the outcomes that hold in a governed organization and is explicit that it does not prescribe risk tolerance. Both operate at institutional altitude and neither specifies platform architecture. The relationship is stated in Section 11: MESA is the institutional instrument, and its Layer 1 is the institutional statement of the invariant this language applies at platform altitude.

---

10. Limitations and Known Failure Modes

A pattern language used without knowing its limits is used on faith. The limits below are stated so that it can be used on diagnosis instead.

10.1 It assumes an institution that values reconstruction above throughput

The language is calibrated to a regulated or self-regulating institution. Where that value does not hold, several patterns lose their justification, because the boundaries they protect are not required by anyone.

This is a scoping statement with a practical consequence. A team that adopts the full catalogue where nobody will ever ask for the evidence has paid the cost of the boundaries and will receive none of the benefit, and it will conclude, reasonably, that the language is overhead. Section 6 exists so that such a team can adopt the subset that pays for itself.

10.2 It governs the architecture, not the models or the practice inside it

The language states what evidence a mechanism shall produce and what constraint it shall respect. It does not state how to train a model, how to evaluate one, how to engineer a prompt, how to run a feature store, or how to decompose a sprint.

The predictable error is to adopt the language and expect it to improve model quality. It will not. It will make the quality question askable, by fixing which model answered under which policy with which evidence, and the answering is somebody else's discipline.

10.3 The abstraction from mechanism to force is a claim, not a guarantee

The catalogue is drawn from the present state of a fast-moving field, and specific mechanisms will change. The patterns are stated at the level of the force each resolves, on the theory that the force outlasts the mechanism. That

doi.org/10.5281/zenodo.22109864

theory is a claim. IP-3 and OP-3 were renamed in this version precisely to reduce the exposure, and the renaming is an admission that the earlier names carried it.

10.4 It assumes a platform team with the authority to set boundaries centrally

Several patterns presume that some body can declare a routing policy, a never-skip test set or a trust tier and have the declaration hold across teams. In a federated organization without that authority, the patterns describe a target state rather than an immediately reachable one, and adoption without first settling the authority question will produce a policy artifact teams route around. The failure is an organizational precondition the language does not supply and cannot substitute for.

10.5 It does not address whether a system should exist

The patterns presume a use case that has already cleared the question of whether the institution should be running it. A workflow deployed against a population that did not consent, governed perfectly under this language, remains a workflow deployed against a population that did not consent. The language will produce a complete and defensible record of it, which is not the same thing as a defence.

10.6 The five characteristic failure modes

A pattern fails not when it is absent but when its boundary is set wrong. Each failure below is a boundary that is present yet mis-set: drifting, ratcheting, missing, proliferating, or local where it should be global.

Router drift. A governed routing policy (IP-1) updated often and reviewed loosely accumulates small changes until sensitive traffic flows somewhere nobody intended. The boundary exists and has stopped being enforced. The defence is the promotion gate of IP-5 applied to the policy itself, not only to the router.

Trust-tier inflation. Authority granted under OP-2 is raised after each success and never lowered after a failure, until agents act with a latitude no evidence justifies. The boundary ratchets in one direction only. The defence is the currency requirement of OP-4.1, so authority can fall as well as rise, and building the demotion path first.

Audit overload. Decision-grade observability (IP-4) and trajectory logging (AP-3) record so much that nothing can be found when needed, and the evidence becomes noise. The boundary on what is recorded is missing. The defence is to record the decision and its basis, not the world.

Boundary explosion. Every incident adds a new boundary until the optimizer has no room left and the platform calcifies. The governance intended to enable autonomy now prevents it. The defence is to treat a boundary as a cost and to retire boundaries through the same governed promotion that adds them, per 2.1.3.

Agent collusion. A population of agents, each satisfying its own capability contract (AP-5) individually, finds a path that violates the institution's intent collectively. No single boundary is crossed, and the gap is between them. The defence is a verifier and policy-enforcement point at the workflow's output (AP-1) and a human-review tier (AP-4) on the composite action, per 5.1.2.

10.7 Where the language has not been tested

Stated plainly, because a specification that omits this section is asking to be trusted rather than examined.

1. **There is no empirical evaluation of any kind.** No pattern has been measured against a control. Every claim about a pattern's benefit is argued from the forces it resolves and the literature it cites.
2. **The unifying scenario is fictional.** Nebula Financial was constructed to exhibit the patterns, which makes it a clear illustration and a weak test. A scenario built to fit a language cannot also refute it.
3. **No pattern reaches evidence level E3.** No implementation unconnected to the author has been evaluated. This is the single largest gap in the evidence base.
4. **The invariant itself is untested.** No study has looked for production mechanisms that resist the boundary-optimizer separation, which is the most direct test the claim admits.
5. **Generalization beyond regulated financial settings is asserted, not shown.**
6. **The composition claim is untested.** That the three artifacts join into one reconstructable account is an architectural assertion. The event identity of 5.2 makes it a testable one, but no measurement exists of how often, in practice, joins fail.
7. **The adoption ordering is a design claim.** That Stage 3 must precede Stage 4 has not been demonstrated by comparison.

doi.org/10.5281/zenodo.22109864

8. **The seventeen are neither exhaustive nor minimal.** Section 3 explains why these survive the boundary test. It does not establish that no eighteenth is required, nor that all seventeen are necessary.

10.8 What would invalidate the language

Falsification is separated into three levels, because refuting one pattern is not refuting the language, and conflating them lets a weak pattern discredit a sound invariant or a sound invariant shelter a weak pattern.

10.8.1 Falsifying the invariant. The invariant fails if either holds:

1. A class of production mechanism is demonstrated that cannot be decomposed into a boundary and an optimizer without loss, which would make 2.1.1 a heuristic rather than a rule.
2. The failure modes of 10.6 are shown to cluster inside individual patterns rather than at the boundary-optimizer confusion the invariant names, which would indicate the invariant is not the organizing principle claimed.

10.8.2 Falsifying individual patterns. A pattern fails if either holds:

1. It satisfies the boundary test of 3.3 and is observably useless in production, which would indicate the admission criterion selects for statability rather than value.
2. Its boundary is shown to be safely relaxable under optimization without the consequence the pattern asserts, which would reclassify it as an optimization under 3.3.2.

10.8.3 Falsifying the composition and sequence. These fail if any holds:

1. Platforms implementing the full applicable set reconstruct decisions no more reliably, or no faster, than platforms implementing none of it, which would remove the case for the catalogue as a whole.
2. A platform reaching Stage 4 without Stage 3 sustains accountability at the same rate as one that did not skip it, which would remove the sequencing requirement of 7.1.
3. The mandatory evidence of 2.3.2 costs more in storage, privacy exposure and engineering time than the reconstruction capability it buys, which would make evidence a recommendation rather than a requirement.

Each is measurable in principle. None has been measured. The licence on this document exists partly so that anyone able to measure one of them may do so without permission.

10.9 Research questions

The specification is written to make the following answerable. They are stated as questions rather than hypotheses because the direction of several is genuinely open.

RQ1. Does explicit boundary-optimizer separation reduce policy violations under adaptive optimization, relative to platforms where constraints and objectives share a mechanism?

RQ2. Does evidence-first architecture reduce mean time to reconstruct an AI decision, and by how much?

RQ3. Does staged authority, bound to a current four-part gate, reduce catastrophic agent actions relative to unrestricted autonomy?

RQ4. What is the storage, privacy and engineering cost of evidence sufficient for reconstruction, and at what point does it exceed the value of the reconstruction?

RQ5. Which patterns are universally applicable and which are environment-dependent, measured against the applicability conditions of 6.1 rather than asserted?

RQ6. Does the three-layer model explain observed production failures better than existing LLMOps or agent-architecture taxonomies?

10.9.1 An open evidence gap. This specification contains no implementation profile from a real deployment. One anonymized profile, naming the environment, the patterns implemented, the before and after state and the evidence available, would materially change the credibility of the catalogue and would move at least one pattern toward E2 or E3. Its absence is recorded here as a known deficiency of version 1.0 rather than left for a reader to notice.

---

doi.org/10.5281/zenodo.22109864

11. Relationship to Other Frameworks

This pattern language operates at platform altitude. The works below operate at other altitudes, and they belong to two families that are named and governed differently.

11.1 Two families, two naming conventions. The **engineering family** consists of this pattern language and the constructs it defines or names: PEVG, PARA, the Boundary Invariant (2.9) and the BOE Declaration (2.10). Every name in that family is **deliberately unmarked**. No trademark is claimed on any of them, and they are offered for use, teaching, assessment and extension by anyone, with attribution, under this document's CC BY 4.0 licence. The **governance family** consists of the frameworks catalogued in the Defensible AI Framework Registry, several of which carry trademarks. A name in this section appearing without a mark is a decision rather than an oversight, and the two families should not be read as one catalogue.

11.2 The Defensible AI Framework Registry. The canonical source for the names, versions, definitions and relationships of the governance-family frameworks. Where this section and the Registry describe the same relationship, **the Registry governs for the governance family and this specification governs for the engineering family.** Stating the precedence is the point: two documents asserting the same relationships will otherwise drift. *Specification: 10.5281/zenodo.22170112.*

The MESA Framework. A four-layer diagnostic model for institutional AI governance, separating a Regulatory Floor, a Strategic Compass, Operational Machinery and a Technical Substrate. The relationship is direct and reciprocal. MESA Layer 1 is the institutional statement of the same invariant: the obligations in the Regulatory Floor are boundaries, satisfied rather than traded against likelihood and impact, and they move only by governance. This language is what MESA Layer 4 looks like when the platform runs on language models. An institution applying MESA to an LLM estate will find that the Technical Substrate's capping rule, under which a control the substrate cannot enforce is not a control, is the institutional form of 2.1.2 [41]. *Specification: https://doi.org/10.5281/zenodo.22109836.*

PEVG: Planner, Executor, Verifier, Generator. The pattern catalogued here as **AP-1**, specified in depth in its own deposit: the four contracts, the verifier's binding judgement on what may be believed, the separate policy-enforcement point of AP-1.1, and the separation's application to AI-assisted work generally rather than to agent workflows alone. Readers requiring the depth treatment should cite that deposit; readers requiring the pattern in its cross-layer context should cite this one. The two names denote one construct, per A.3. No trademark is claimed on the name. *Specification: 10.5281/zenodo.22170132.*

PARA: Perception, Action, Reasoning, Adaptation. The pattern catalogued here as **OP-5**, specified in depth in its own deposit: the four faculties, their distinct authority types, and the registry entry that turns a faculty into a contract. The fourth faculty is **Adaptation**; any source rendering it as Reflection is in error. No trademark is claimed on the name. *Specification: 10.5281/zenodo.22170139.*

The Five-Gate Deployment Model™. A deployment decision structure of five checkpoints between an approved idea and production, each carrying entry criteria, an approval authority, a required evidence set, exit criteria and a defined rejection path. Its relationship is at OP-2 and OP-4: the gate structure is where a trust-tier increase acquires a decision consequence, and where a benchmark result becomes something other than a number in a report. Each gate's evidence record is an instance of the BOE Declaration, per 2.10.3, which is what makes the two specifications compose rather than merely coexist. *Specification: 10.5281/zenodo.22170122.*

The source texts. The seventeen patterns are developed at chapter length across the three volumes of The Full-Stack AI Engineering Series [35][36][37], with the chapter mapping in Appendix A. This document is a standalone, citable restatement written for a reader with access to neither the books nor the working paper.

Appendix A. Cross-Corpus Naming Crosswalk (Normative)

The seventeen constructs are specified in three places: this specification, the author's working papers, and the companion book series. Several carry different names in each. A reader arriving from one cannot presently find the others. This appendix fixes the equivalences.

A.1 The identifier of the form IP-n, AP-n or OP-n is the canonical reference for each construct and shall be used in preference to any name where precision is required.

doi.org/10.5281/zenodo.22109864

A.2 Where names differ across the three sources, all are valid names for the same construct and none is deprecated. Two patterns were renamed in this specification at the force level, for the durability reason stated in 10.3; the prior names remain valid and are recorded below.

ID	Name in this specification	Name in the papers	Name or acronym in the books	Developed at length in
IP-1	Governed Routing Policy	Governed Routing Policy	**The Routing Brain**, with the versioned routing policy artifact. The Routing Brain is the optimizer half; the policy artifact is the boundary half	Book 1, Ch. 4
IP-2	Tiered Model Pool	Tiered Model Pool	The model pool as operational tiers: small, mid-tier, frontier	Book 1, Ch. 2
IP-3	**Adaptive Latency-Bounded Serving** *(renamed)*	SLO-Aware Speculative Decoding	Speculative decoding as an SRE primitive	Book 1, Ch. 6
IP-4	Decision-Grade Observability	Decision-Grade Observability	Observability for LLM routers and models; the routing-event log schema	Book 1, Ch. 8
IP-5	Router Lifecycle Security	Router Lifecycle Security	Router lifecycle security and governance	Book 1, Ch. 9
AP-1	**Planner, Executor, Verifier, Generator**	Planner, Executor, Verifier, Generator	**PEVG.** The acronym appears in neither paper	**Book 2, Ch. 3**
AP-2	**Activity-on-Vertex Workflow Graph**	Activity-on-Vertex Workflow Graph	**AOV graph**, also the agent graph. The abbreviation appears in neither paper	**Book 2, Ch. 5**
AP-3	Structure-Only Trajectory Logging	Structure-Only Trajectory Logging	**The trajectory record**, and its governance as retained data	Book 2, Ch. 4
AP-4	Tiered Human-in-the-Loop	Tiered Human-in-the-Loop	Human-in-the-loop as an orchestration layer	Book 2, Ch. 8
AP-5	Capability Contract	Capability Contract	Capability Contract. Identical in all three	Book 2, Ch. 7 and 12
OP-1	Golden Path	Golden Path	Golden paths and paved roads. Identical in all three	Book 3, Ch. 3
OP-2	Policy-Bounded Agentic Delivery	Policy-Bounded Agentic Delivery	**Delivery Guardrail** (boundary artifact) and **trust tier** (authority scale)	Book 3, Ch. 4 and 5
OP-3	**Risk-Bounded Adaptive Test Selection** *(renamed)*	Reinforcement-Framed Adaptive Testing	Reinforcement-learned adaptive testing	Book 3, Ch. 6
OP-4	Benchmark Before Authority	Benchmark Before Authority	Evaluation and benchmarking of DevOps agents	Book 3, Ch. 7
OP-5	**Perception, Action, Reasoning, Adaptation**	Perception, Action, Reasoning, Adaptation	**PARA**, the PARA operating model	**Book 3, Ch. 8**
OP-6	Per-Team Cost Attribution	Per-Team Cost Attribution	Cost attribution within the FinOps layer	Book 3, Ch. 10
OP-7	Budget as Boundary	Budget as Boundary	No corresponding named construct. Added after the series was drafted	Papers and this specification only

A.3 The three principal bridges. Three equivalences carry most of the citation traffic, because in each case an acronym that is load-bearing in the books is entirely absent from the papers.

- **AP-1 = PEVG.** Planner, Executor, Verifier, Generator. A planner that decomposes the task, an executor that performs tool actions under contract, a verifier that holds binding judgement on what may be believed, and a generator that produces the final response from what survived the check.
- **AP-2 = AOV graph.** Activity-on-Vertex Workflow Graph. Nodes are agent invocations, edges are dependencies, and the work lives in the vertices rather than on the edges.
- **OP-5 = PARA.** Perception, Action, Reasoning, Adaptation. The acronym appears once in the working paper and not at all in the magazine version. The fourth faculty is **Adaptation**. It is not Reflection, and any source rendering it as Reflection is in error.

doi.org/10.5281/zenodo.22109864

A.4 A book construct with no pattern identifier. The **Reasoning Profile**, specified in Book 2 Chapter 4, is a set of per-business-line dials governing how a planner decomposes a task, including maximum step count, exploration appetite, retry budget and downstream verifier strictness. It is a parameterization of AP-1's planner role rather than a pattern, because it fails the boundary test of 3.3, as recorded in the derivation table of 3.4. It is named here so that a reader encountering it in the books does not search the papers for a pattern that was never there.

A.5 A pattern with no book counterpart. OP-7 Budget as Boundary was added during a revision after the series manuscripts were complete. A reader working from the books alone will find the cost-attribution half of the economic loop, in Book 3 Chapter 10, and will not find the spend-cap-as-boundary half.

Appendix B. Publication Provenance

This appendix exists because the record is easy to misread, and a citation to a preprint that does not exist would be worse than no citation at all.

B.1 The working paper. *A Pattern Language for Production LLM Platforms: Governed Routing, Agent Orchestration, and AI-Native Delivery in Regulated Environments* [38] is a working draft, revised through a third finalization pass in July 2026. It has **not** been submitted to any preprint server, has **not** been peer reviewed, and carries **no** arXiv identifier or DOI.

B.2 The magazine version. *The Boundary and the Optimizer: A Pattern Language for Production LLM Platforms* [39] is a condensed version formatted to IEEE Software magazine limits. Its running head carries a self-applied label reading ”submitted version”. That label is a formatting artifact of the draft and is **not** evidence that a submission was lodged or accepted. No acceptance record, DOI or publication entry exists for it.

B.3 The public exposition. A prose exposition is published at the author's site [40], where the full-text working paper is distributed under CC BY-NC-ND 4.0. That licence governs the working paper PDF. It does **not** govern this specification, which is CC BY 4.0.

B.4 The books. The seventeen patterns are developed at chapter length across the three volumes of The Full-Stack AI Engineering Series: Book 1, *LLM Systems in Production*, ISBN 978-1-0678317-1-4; Book 2, *Prompt Systems and Agent Orchestration*, ISBN 978-1-0678317-2-1; Book 3, *DevOps for AI-Native Platforms*, ISBN 978-1-0678317-3-8.

B.5 Zenodo relations. This record carries **no** `isPreviousVersionOf` and **no** `isSupplementTo` relation to any preprint identifier, because no such identifier exists. Should the working paper later be submitted and assigned one, the relation may be added then.

B.6 Trademark. No trademark is claimed on the invariant, on any of the seventeen pattern names, or on the acronyms PEVG and PARA. The CC BY 4.0 licence applies to the text of this specification and may be exercised without further permission.

B.7 Relationship to the pre-publication draft. An earlier internal draft of this specification, dated 26 August 2026 and not deposited, organized the material differently: related work preceded the catalogue, provenance sat in the front matter, and the applicability rule required all seventeen patterns of every regulated institution. That draft was never published and carries no identifier. Version 1.0 is the first public version.

---

12. References

Every citation below was verified against the publisher record, the ACM or ACL digital library, or the arXiv abstract record on 26 August 2026. Sources without a citable individual author are attributed to their publishing organization. Where a source's title differs from a rendering elsewhere in the author's corpus, the title recorded here is the one carried by the publisher record.

The pattern tradition

[1] Alexander, C., Ishikawa, S., and Silverstein, M. *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press, New York, 1977. ISBN 0-19-501919-9. With Max Jacobson, Ingrid Fiksdahl-King and Shlomo Angel; Center for Environmental Structure Series, Volume 2.

doi.org/10.5281/zenodo.22109864

[2] Alexander, C. *The Timeless Way of Building*. Oxford University Press, New York, 1979. ISBN 0-19-502402-8. Center for Environmental Structure Series, Volume 1.

[3] Gamma, E., Helm, R., Johnson, R., and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading, MA, 1994. ISBN 0-201-63361-2.

[4] Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., and Stal, M. *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns*. John Wiley and Sons, Chichester, 1996. ISBN 0-471-95869-7.

Foundation-model and agent pattern catalogues

[5] Liu, Y., Lo, S. K., Lu, Q., Zhu, L., Zhao, D., Xu, X., Harrer, S., and Whittle, J. Agent Design Pattern Catalogue: A Collection of Architectural Patterns for Foundation Model based Agents. arXiv:2405.10467, May 2024. https://arxiv.org/abs/2405.10467

[6] Cai, Y., Li, R., Liang, P., Shahin, M., and Li, Z. Designing LLM-based Multi-Agent Systems for Software Engineering Tasks: Quality Attributes, Design Patterns and Rationale. arXiv:2511.08475, November 2025. https://arxiv.org/abs/2511.08475

[7] Vandeputte, F. Foundational Design Principles and Patterns for Building Robust and Adaptive GenAI-Native Systems. In *Proceedings of the 2025 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '25)*, Singapore, October 2025, pages 44-62. DOI: 10.1145/3759429.3762620. Preprint: arXiv:2508.15411.

[8] Gulli, A. *Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems*. Springer Nature Switzerland, 2025. ISBN 978-3-032-01401-6 (softcover); eBook ISBN 978-3-032-01402-3; DOI 10.1007/978-3-032-01402-3.

Machine-learning systems and platform engineering

[9] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. Hidden Technical Debt in Machine Learning Systems. In *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, pages 2503-2511.

[10] Kreuzberger, D., Kuhl, N., and Hirschl, S. Machine Learning Operations (MLOps): Overview, Definition, and Architecture. *IEEE Access*, vol. 11, pages 31866-31879, 2023. DOI: 10.1109/ACCESS.2023.3262138.

[11] Niemen, G. How We Use Golden Paths to Solve Fragmentation in Our Software Ecosystem. Spotify Engineering Blog, 17 August 2020. https://engineering.atspotify.com/2020/08/how-we-use-golden-paths-to-solve-fragmentation-in-our-software-ecosystem

Governance standards

[12] ISO/IEC 42001:2023, *Information technology - Artificial intelligence - Management system*. International Organization for Standardization and International Electrotechnical Commission, Geneva, 2023. Catalogue entry: https://www.iso.org/standard/42001

[13] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. Gaithersburg, MD, January 2023. https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf

Routing and serving

[14] Jitkrittum, W., Narasimhan, H., Rawat, A. S., Juneja, J., Wang, C., Wang, Z., Go, A., Lee, C.-Y., Shenoy, P., Panigrahy, R., Menon, A. K., and Kumar, S. Universal Model Routing for Efficient LLM Inference. arXiv:2502.08773, February 2025.

[15] Jain, K., Parayil, A., Mallick, A., Choukse, E., Qin, X., Zhang, J., Goiri, I., Wang, R., Bansal, C., Ruhle, V., Kulkarni, A., Kofsky, S., and Rajmohan, S. Intelligent Router for LLM Workloads: Improving Performance Through Workload-Aware Load Balancing. arXiv:2408.13510, August 2024.

[16] Chuang, Y.-N., Yu, L., Wang, G., Zhang, L., Liu, Z., Cai, X., Sui, Y., Braverman, V., and Hu, X. Confident or Seek Stronger: Exploring Uncertainty-Based On-device LLM Routing From Benchmarking to Generalization. arXiv:2502.04428, February 2025.

[17] Li, B., Jiang, Y., Gadepally, V., and Tiwari, D. LLM Inference Serving: Survey of Recent Advances and Opportunities. arXiv:2407.12391, July 2024.

doi.org/10.5281/zenodo.22109864

[18] Huang, K., Wu, H., Shi, Z., Zou, H., Yu, M., and Shi, Q. AdaSpec: Adaptive Speculative Decoding for Fast, SLO-Aware Large Language Model Serving. arXiv:2503.05096, March 2025. Accepted at ACM SoCC 2025.

[19] Sadhukhan, R., Chen, J., Chen, Z., Tiwari, V., Lai, R., Shi, J., Yen, I. E.-H., May, A., Chen, T., and Chen, B. MagicDec: Breaking the Latency-Throughput Tradeoff for Long Context Generation with Speculative Decoding. arXiv:2408.11049, August 2024.

[20] Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 7655-7671. https://aclanthology.org/2024.findings-acl.456/

[21] Lin, Q., Ji, X., Zhai, S., Shen, Q., Zhang, Z., Fang, Y., and Gao, Y. Life-Cycle Routing Vulnerabilities of LLM Router. arXiv:2503.08704, March 2025.

Agent orchestration

[22] Li, Z., Zhang, H., Han, S., Liu, S., Xie, J., Zhang, Y., Choi, Y., Zou, J., and Lu, P. In-the-Flow Agentic System Optimization for Effective Planning and Tool Use. arXiv:2510.05592, October 2025. The framework is named AgentFlow.

[23] Arora, D., Sonwane, A., Wadhwa, N., Mehrotra, A., Utpala, S., Bairi, R., Kanade, A., and Natarajan, N. MASAI: Modular Architecture for Software-engineering AI Agents. arXiv:2406.11638, June 2024.

[24] Niu, B., Song, Y., Lian, K., Shen, Y., Yao, Y., Zhang, K., and Liu, T. Flow: Modularized Agentic Workflow Automation. *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2501.07834. *Note: this work defines workflows as activity-on-vertex graphs, which is the published grounding for AP-2.*

[25] Zhang, J., Xiang, J., Yu, Z., Teng, F., Chen, X., Chen, J., Zhuge, M., Cheng, X., Hong, S., Wang, J., Zheng, B., Liu, B., Luo, Y., and Wu, C. AFlow: Automating Agentic Workflow Generation. *International Conference on Learning Representations (ICLR)*, 2025 (Oral). arXiv:2410.10762. https://openreview.net/forum?id=z5uVAKwmjf

[26] Kaidhapuram, S. R. Human-in-the-Loop (HITL) Orchestration for Agentic Use-Cases: A Practical Framework for Supervising Autonomous AI Agents in Production Environments. *International Journal of Computer Techniques*, vol. 12, no. 6, 2025. *Note: this is a low-visibility venue and is cited alongside [27] rather than alone.*

[27] Mosqueira-Rey, E., Hernandez-Pereira, E., Alonso-Rios, D., Bobes-Bascaran, J., and Fernandez-Leal, A. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*, vol. 56, no. 4, pages 3005-3054, 2023. DOI: 10.1007/s10462-022-10246-w.

AI-native delivery

[28] Pitkar, H. Platform Engineering and Developer Experience: A Systematic Review of Concepts, Benefits and Future Directions. *World Journal of Advanced Engineering Technology and Sciences*, vol. 18, no. 2, pages 241-248, 2026. DOI: 10.30574/wjaets.2026.18.2.0112.

[29] Baqar, M., Naqvi, S., and Khanda, R. AI-Augmented CI/CD Pipelines: From Code Commit to Production with Autonomous Decisions. arXiv:2508.11867, August 2025.

[30] Joshi, S. A Review of Generative AI and DevOps Pipelines: CI/CD, Agentic Automation, MLOps Integration, and Large Language Models. *SSRN Electronic Journal*, 2025. SSRN identifier 5290005.

[31] Soni, A. A., Parikh, M., Dhenia, R. N. K., Soni, J. A., Jha, A. R., and Shah, S. M. Reinforcement Learning for Dynamic Workflow Optimization in CI/CD Pipelines. arXiv:2601.11647, January 2026.

[32] Nabil, R. L., Zhu, H.-N., and Rubio-Gonzalez, C. CI-Bench: A Framework for Evaluating Large Language Model Tools on CI Failures. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Companion (ICSE-Companion '26), Demonstrations*, Rio de Janeiro, Brazil, April 2026.

[33] Ghaleb, T. A. When AI Agents Touch CI/CD Configurations: Frequency and Success. arXiv:2601.17413, January 2026.

[34] Song, J. The Second Half of Cloud Native: The Era of AI-Native Platform Engineering Has Arrived. 2025. https://jimmysong.io/blog/cloud-native-second-half-ai-native-platform-engineering/ *Note: this is a practitioner source cited for the established perception-action framing that OP-5 extends, not for the four-faculty model itself.*

Source volumes and companion work

doi.org/10.5281/zenodo.22109864

[35] Khan, N. A. *LLM Systems in Production: Cloud-Native Patterns for AI Engineers*. The Full-Stack AI Engineering Series, Book 1. iSystematic Inc., 2026. ISBN 978-1-0678317-1-4 (paperback); 978-1-0678317-5-2 (hardcover). Patterns IP-1 to IP-5 are developed here.

[36] Khan, N. A. *Prompt Systems and Agent Orchestration*. The Full-Stack AI Engineering Series, Book 2. iSystematic Inc., 2026. ISBN 978-1-0678317-2-1 (paperback); 978-1-0678317-6-9 (hardcover). Patterns AP-1 to AP-5 are developed here; PEVG is Chapter 3.

[37] Khan, N. A. *DevOps for AI-Native Platforms*. The Full-Stack AI Engineering Series, Book 3. iSystematic Inc., 2026. ISBN 978-1-0678317-3-8 (paperback); 978-1-0678317-7-6 (hardcover). Patterns OP-1 to OP-6 are developed here; PARA is Chapter 8.

[38] Khan, N. A. *A Pattern Language for Production LLM Platforms: Governed Routing, Agent Orchestration, and AI-Native Delivery in Regulated Environments*. Working paper, 2026. Not submitted; not peer reviewed; no preprint identifier assigned at the date of this document. See Appendix B.

[39] Khan, N. A. *The Boundary and the Optimizer: A Pattern Language for Production LLM Platforms*. Condensed magazine draft, 2026. Unsubmitted at the date of this document; no publication record. See Appendix B.

[40] A Pattern Language for Production LLM Platforms. Reference page: https://nabeelkhan.com/pattern-language-production-llm-platforms

[41] The MESA Framework. Reference page: https://nabeelkhan.com/frameworks/mesa

---

Document History

Version	Date	Change
1.1	30 August 2026	Names the Boundary Invariant (2.9) and the BOE Declaration (2.10). No normative change to any pattern, to the conceptual model, or to the conformance language. Both names are deliberately unmarked and are offered for use with attribution under this document's licence. Section 11 gains the two-family naming convention and the precedence rule that resolves a disagreement with the Defensible AI Framework Registry. The citation identifier moves to the concept DOI, which resolves to the latest version, and the figure attribution footers carry it
1.0	26 August 2026	First public specification. Restructured so that the conceptual model and catalogue precede related work. Adds a pattern derivation method with an explicit rejection rule and worked rejections (Section 3), evidence levels E0 to E3 with the finding that no pattern reaches E3 (3.5), a grounding taxonomy (3.6), a boundary taxonomy of five classes (2.5), a pattern relationship graph (3.8), a normative event identity for evidence correlation (5.2), applicability conditions replacing a categorical requirement (Section 6), self-declared conformance with a published profile (Section 8), three-level falsification (10.8) and six research questions (10.9). IP-3 and OP-3 renamed at the force level, with prior names retained in Appendix A. AP-1 gains the epistemic and operational boundary distinction; AP-4 requires a decision-risk signal rather than raw model confidence; OP-4 requires a four-part eligibility gate rather than a benchmark score; OP-7 is stated as constrained optimization. Provenance moved to Appendix B

About the Author

Nabeel Khan is an enterprise AI architect and governance advisor, and Founder & Solutions Architect at iSystematic Inc. He has spent twenty-five years architecting enterprise data and machine-learning systems in government, healthcare, financial services and telecommunications across three continents, grounded in TOGAF, DMBOK, ISO 27001 and SOC 2.

He works on governed production AI; systems whose authority is bounded by the architecture, whose decisions carry the evidence needed to defend them, and whose behaviour the institution can explain afterwards. The

doi.org/10.5281/zenodo.22109864

frameworks specified here, and the regulatory knowledge behind them, are published as machine-accessible AI expertise and are callable over MCP without a key at mcp.nabeelkhan.com/api/mcp, so a reader may query this specification's contents directly and receive answers carrying their source.

ORCID 0009-0005-5364-914X · nabeelkhan.com · Canada

Licence

© 2026 Nabeel Khan. This work is licensed under a Creative Commons Attribution 4.0 International licence (CC BY 4.0). It may be copied, redistributed, adapted and built upon for any purpose, including commercially, provided appropriate credit is given, a link to the licence is provided, and changes are indicated. Full licence text: https://creativecommons.org/licenses/by/4.0/

The licence covers the text of this specification only. It does not extend to the working paper [38], which is distributed separately under CC BY-NC-ND 4.0, nor to the source volumes [35][36][37], which are separately licensed.

How to attribute this work

This work is licensed CC BY 4.0. Section 3(a)(1)(A) of that licence requires attribution in the manner the licensor requests, and Section 3(a)(1)(C) requires a link to the material where practicable. This section is that request. Please use one of the following.

Short credit, for slides, diagrams and internal documents:

```
A Pattern Language for Production LLM Platforms by Nabeel Khan ·
  https://nabeelkhan.com/frameworks/pattern-language · CC BY 4.0
```

Inline credit, for web pages, articles and product documentation:

```
Based on <a href="https://nabeelkhan.com/frameworks/pattern-language">A
  Pattern Language for Production LLM Platforms</a> by Nabeel Khan,
licensed under <a href="https://creativecommons.org/licenses/by/4.0/">CC
  BY 4.0</a>.
```

Academic citation:

```
Khan, N. A. (2026). A Pattern Language for Production LLM Platforms:
  Governed Routing, Agent Orchestration, and AI-Native Delivery.
(Version 1.1). Zenodo. https://doi.org/10.5281/zenodo.22109864
```

The DOI given is the concept DOI and always resolves to the latest version. Cite it unless you need to pin one specific version.

If you have adapted the work, say that changes were made, and do not imply that this author or iSystematic Inc. endorses your version.

Trademarks. No trademark is claimed on this pattern language or on any pattern name in it. The names are offered for use without restriction beyond attribution.